THE BELOW IS A SCRIPT TO PARSE, Code verify, and determine bugs and real true apps with macOS i'm gonna paste another one below it in this entire purpose is to turn on a TCC dump a pee, cap dump record the terminal stage the evidence run the script output to a specific file picking up all the network traffic with the TCC dump in the P camp to zip it up for a beautiful bug bounty, but all the files in the folder directories need to be searched recursively to accurately code verified debug and do everything necessary to the output folder to get what we need!!!! so I want you to take both of these and enhance them and make them fucking five times better than what they are to wear. A bug bunny could not be eating undeniable truth just go with me because I made it that damn easy for them

```
#!/bin/bash

set -euo pipefail


# atlas_submission_capture.sh

#

# Purpose:

#   Build a short, repeatable, reviewer-friendly evidence packet for Atlas and

#   any clone-app comparisons using the same workflow.

#

# Key requirement:

#   This script DOES recurse through app directories. It does not stop at the

#   head of a directory. It hunts through the bundle for:

#     - executables

#     - nested .app bundles

#     - Info.plist and other plist files

#     - dylibs / frameworks / xpc services

#     - helper apps

#     - provisioning profiles
```

```
#     - launchd-related plists inside the bundle
#
# What it captures per app pair:
#   - suspect outer launcher metadata, hashes, trust results
#   - baseline outer launcher metadata, hashes, trust results
#   - recursive inventory of important bundle artifacts
#   - nested runtime discovery + validation
#   - interesting Frameworks payload hashes (e.g. OwlBridge.dylib)
#   - live process captures if the app is running
#   - user TCC rows for discovered bundle ids
#   - filtered tccd logs for recent runtime activity
#   - optional PCAP hashes if you place captures next to the app case
#   - per-app markdown executive summary
#
# What it does NOT do:
#   - no sudo
#   - no writes to app bundles
#   - no quarantine stripping
#   - no execution from suspect bundles
#   - no Microsoft / Google / MDM / Recovery scope creep
#
# ------------------------------------------------------------------------
# MANIFEST FORMAT
# ------------------------------------------------------------------------
# Provide a CSV with this header:
# name,suspect_app,baseline_app,process_match,pcap_glob,extra_glob
```

```
#
# Example:
# name,suspect_app,baseline_app,process_match,pcap_glob,extra_glob
# Atlas,/Applications/ChatGPT Atlas.app,/Applications/ChatGPT Atlas 2.app,"ChatGPT Atlas","/Volumes/Ellis/Investigations/pcaps/*ATLAS*.cap","/Volumes/Ellis/Investigations/atlas_helpers/*"
# Codex,/Applications/Codex.app,/Applications/Codex 2.app,Codex,,
# Chrome,/Applications/Google Chrome.app,/Applications/Google Chrome Fresh.app,"Google Chrome",,
#
# Run:
#   chmod +x atlas_submission_capture.sh
#   ./atlas_submission_capture.sh manifest.csv /Volumes/Ellis/Investigations
#
# Results:
#   /Volumes/Ellis/Investigations/atlas_submission_capture_<UTC>/...
# ------------------------------------------------------------------------

if [[ $# -lt 1 || $# -gt 2 ]]; then
  echo "Usage: $0 <manifest.csv> [output_base]"
  exit 1
fi

MANIFEST="$1"
OUT_BASE="${2:-$PWD}"

if [[ ! -f "$MANIFEST" ]]; then
  echo "[FATAL] Manifest not found: $MANIFEST"
```

```
  exit 1
fi

mkdir -p "$OUT_BASE"
OUT_BASE="$(cd "$OUT_BASE" && pwd -P)"
RUN_TS="$(date -u +%Y%m%dT%H%M%SZ)"
RUN_DIR="$OUT_BASE/atlas_submission_capture_${RUN_TS}"
mkdir -p "$RUN_DIR"

LOG="$RUN_DIR/run.log"
SUMMARY="$RUN_DIR/summary.tsv"
ARTIFACT_SUMMARY="$RUN_DIR/artifact_summary.tsv"
printf 'app\trole\tpath\tsha256\tsize\tbirth\tmtime\tbundle_id\tteam_id\tcodesign_status\tspctl_status\tnotes\n' > "$SUMMARY"
printf 'app\trole\tartifact_type\tpath\tsha256\tsize\tbirth\tmtime\tbundle_id\tteam_id\tnotes\n' > "$ARTIFACT_SUMMARY"

log() {
  printf '[%s] %s\n' "$(date -u +%Y-%m-%dT%H:%M:%SZ)" "$*" | tee -a "$LOG"
}

trim() {
  local s="$1"
  s="${s#${s%%[![:space:]]*}}"
  s="${s%${s##*[![:space:]]}}"
  printf '%s' "$s"
```

```
}

csv_get() {
  local line="$1" idx="$2"
  awk -F',' -v i="$idx" '{gsub(/^"|"$/, "", $i); print $i}' <<< "$line"
}

safe_name() {
  printf '%s' "$1" | tr -cs 'A-Za-z0-9._-' '_'
}

sha256_file() {
  local f="$1"
  shasum -a 256 "$f" | awk '{print $1}'
}

file_size() {
  stat -f '%z' "$1" 2>/dev/null || echo ""
}

birth_time() {
  stat -f '%SB' -t '%Y-%m-%dT%H:%M:%SZ' "$1" 2>/dev/null || echo ""
}

mtime_time() {
```

```
  stat -f '%Sm' -t '%Y-%m-%dT%H:%M:%SZ' "$1" 2>/dev/null || echo ""
}

bundle_id_of_app() {
  local app="$1"
  /usr/libexec/PlistBuddy -c 'Print :CFBundleIdentifier' "$app/Contents/Info.plist" 2>/dev/null || true
}

bundle_executable_of_app() {
  local app="$1"
  /usr/libexec/PlistBuddy -c 'Print :CFBundleExecutable' "$app/Contents/Info.plist" 2>/dev/null || true
}

capture_codesign() {
  local target="$1" out="$2"
  if codesign -dv --verbose=4 "$target" > "$out" 2>&1; then
    echo "ok"
  else
    echo "fail"
  fi
}

capture_spctl() {
  local target="$1" out="$2"
  if spctl -a -vv "$target" > "$out" 2>&1; then
    echo "accepted"
```

```
  else
    echo "rejected"
  fi
}

extract_team_id() {
  local f="$1"
  awk -F= '/^TeamIdentifier=/{print $2; exit}' "$f" | tr -d '[:space:]'
}

extract_identifier() {
  local f="$1"
  awk -F= '/^Identifier=/{print $2; exit}' "$f" | tr -d '[:space:]'
}

is_macho_executable() {
  local f="$1"
  file "$f" 2>/dev/null | grep -Eq 'Mach-O.*(executable|dynamically linked shared library|bundle)'
}

add_artifact_summary() {
  local app_name="$1" role="$2" artifact_type="$3" path="$4" sha="$5" size="$6" birth="$7" mtime="$8"
bundle_id="$9" team_id="${10}" notes="${11}"
  printf '%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\n' \
    "$app_name" "$role" "$artifact_type" "$path" "$sha" "$size" "$birth" "$mtime" "$bundle_id" "$team_id"
"$notes" >> "$ARTIFACT_SUMMARY"
```

```
}

capture_path_artifact() {
  local app_name="$1" role="$2" artifact_type="$3" path="$4" out_dir="$5"
  mkdir -p "$out_dir"
  printf '%s\n' "$path" > "$out_dir/path.txt"

  local sha="" size="" birth="" mtime="" bundle_id="" team_id="" notes=""
  if [[ -f "$path" ]]; then
    sha="$(sha256_file "$path" 2>/dev/null || true)"
    size="$(file_size "$path")"
    birth="$(birth_time "$path")"
    mtime="$(mtime_time "$path")"
    file "$path" > "$out_dir/file.txt" 2>&1 || true
    stat "$path" > "$out_dir/stat.txt" 2>&1 || true
    xattr -lr "$path" > "$out_dir/xattr.txt" 2>&1 || true
    shasum -a 256 "$path" > "$out_dir/sha256.txt" 2>&1 || true

    if is_macho_executable "$path"; then
      otool -L "$path" > "$out_dir/otool_L.txt" 2>&1 || true
      otool -l "$path" > "$out_dir/otool_l.txt" 2>&1 || true
      codesign -dv --verbose=4 "$path" > "$out_dir/codesign.txt" 2>&1 || true
      bundle_id="$(extract_identifier "$out_dir/codesign.txt")"
      team_id="$(extract_team_id "$out_dir/codesign.txt")"
    fi
```

```
    if [[ "$artifact_type" == "plist" || "$path" == *.plist ]]; then
      plutil -p "$path" > "$out_dir/plutil.txt" 2>&1 || true
    fi
  elif [[ -d "$path" ]]; then
    stat "$path" > "$out_dir/stat.txt" 2>&1 || true
    xattr -lr "$path" > "$out_dir/xattr.txt" 2>&1 || true
    notes="directory"
  else
    notes="missing"
  fi

  add_artifact_summary "$app_name" "$role" "$artifact_type" "$path" "$sha" "$size" "$birth" "$mtime"
"$bundle_id" "$team_id" "$notes"
}

capture_static_bundle() {
  local app_name="$1"
  local role="$2"
  local app_path="$3"
  local case_dir="$4"

  if [[ ! -d "$app_path" ]]; then
    log "[WARN] Missing $role app for $app_name: $app_path"
    return 0
  fi
```

```
local role_dir="$case_dir/${role}"
mkdir -p "$role_dir"

local exec_name exec_path sha size birth mtime bundle_id codesign_state spctl_state team_id ident
exec_name="$(bundle_executable_of_app "$app_path")"
bundle_id="$(bundle_id_of_app "$app_path")"
exec_path="$app_path/Contents/MacOS/$exec_name"

printf '%s\n' "$app_path" > "$role_dir/app_path.txt"
printf '%s\n' "$bundle_id" > "$role_dir/bundle_id.txt"
printf '%s\n' "$exec_name" > "$role_dir/executable_name.txt"

if [[ -f "$exec_path" ]]; then
  sha="$(sha256_file "$exec_path")"
  size="$(file_size "$exec_path")"
  birth="$(birth_time "$exec_path")"
  mtime="$(mtime_time "$exec_path")"
  file "$exec_path" > "$role_dir/file.txt" 2>&1 || true
  stat "$exec_path" > "$role_dir/stat.txt" 2>&1 || true
  xattr -lr "$exec_path" > "$role_dir/xattr.txt" 2>&1 || true
  otool -L "$exec_path" > "$role_dir/otool_L.txt" 2>&1 || true
  otool -l "$exec_path" > "$role_dir/otool_l.txt" 2>&1 || true
  shasum -a 256 "$exec_path" > "$role_dir/sha256.txt" 2>&1 || true
else
  sha=""
```

```
    size=""
    birth=""
    mtime=""
    printf '[WARN] Missing executable: %s\n' "$exec_path" > "$role_dir/file.txt"
  fi

  codesign_state="$(capture_codesign "$app_path" "$role_dir/codesign_bundle.txt")"
  spctl_state="$(capture_spctl "$app_path" "$role_dir/spctl_bundle.txt")"

  if [[ -f "$exec_path" ]]; then
    capture_codesign "$exec_path" "$role_dir/codesign_executable.txt" >/dev/null ll true
  fi

  ident="$(extract_identifier "$role_dir/codesign_executable.txt")"
  team_id="$(extract_team_id "$role_dir/codesign_executable.txt")"

  printf '%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\n' \
    "$app_name" "$role" "$app_path" "$sha" "$size" "$birth" "$mtime" "$bundle_id" "$team_id"
"$codesign_state" "$spctl_state" "$ident" >> "$SUMMARY"

  printf '%s\n' "$bundle_id" >> "$case_dir/bundle_ids.all"
  printf '%s\n' "$ident" >> "$case_dir/code_identifiers.all"

  add_artifact_summary "$app_name" "$role" "app_bundle" "$app_path" "" "" "" "" "$bundle_id" "$team_id"
"$ident"
  if [[ -f "$exec_path" ]]; then
    add_artifact_summary "$app_name" "$role" "outer_executable" "$exec_path" "$sha" "$size" "$birth"
```

```
"$mtime" "$bundle_id" "$team_id" "$ident"
  fi
}

recursive_inventory_bundle() {
  local app_name="$1"
  local role="$2"
  local app_path="$3"
  local case_dir="$4"
  local inv_dir="$case_dir/${role}_inventory"
  mkdir -p "$inv_dir"

  local index=0
  while IFS= read -r p; do
    [[ -z "$p" ]] && continue
    index=$((index+1))
    local base stem out artifact_type
    base="$(basename "$p")"
    stem="$(printf '%05d_%s' "$index" "$(safe_name "$base")")"
    out="$inv_dir/$stem"

    artifact_type="file"
    if [[ -d "$p" && "$p" == *.app ]]; then artifact_type="nested_app"; fi
    if [[ -d "$p" && "$p" == *.xpc ]]; then artifact_type="xpc_service"; fi
    if [[ -f "$p" && "$p" == *.plist ]]; then artifact_type="plist"; fi
    if [[ -f "$p" && "$p" == *.dylib ]]; then artifact_type="dylib"; fi
```

```
    if [[ -d "$p" && "$p" == *.framework ]]; then artifact_type="framework"; fi
    if [[ -f "$p" && "$p" == *embedded.provisionprofile ]]; then artifact_type="provisioning_profile"; fi

    capture_path_artifact "$app_name" "$role" "$artifact_type" "$p" "$out"
  done < <(
    find "$app_path" \(
      -type d \( -name '*.app' -o -name '*.framework' -o -name '*.xpc' \) -o \
      -type f \( -name '*.plist' -o -name '*.dylib' -o -name 'embedded.provisionprofile' -o -perm -111 \)
    \) | sort
  )
}

capture_framework_payload_strings() {
  local app_path="$1"
  local inv_dir="$2"
  [[ -d "$inv_dir" ]] || return 0
  while IFS= read -r file; do
    [[ -z "$file" ]] && continue
    if [[ -f "$file/path.txt" ]]; then
      local p
      p="$(cat "$file/path.txt")"
      if [[ -f "$p" && ( "$p" == *.dylib || "$p" == *OwlBridge* || "$p" == *Bridge* ) ]]; then
        strings "$p" | grep -Ei 'owlbridge|icloud|keychain|controller|dyld|bridge' > "$file/interesting_strings.txt"
2>/dev/null || true
      fi
    fi
```

```
  done < <(find "$inv_dir" -type d | sort)
}

capture_nested_runtimes() {
  local app_name="$1"
  local role="$2"
  local app_path="$3"
  local case_dir="$4"
  local nested_dir="$case_dir/${role}_nested"
  mkdir -p "$nested_dir"

  local nested index=0
  while IFS= read -r nested; do
    [[ -z "$nested" ]] && continue
    [[ "$nested" == "$app_path" ]] && continue
    index=$((index+1))
    capture_static_bundle "$app_name" "${role}_nested_${index}" "$nested" "$case_dir"
  done < <(find "$app_path" -type d -name '*.app' | sort)
}

capture_extra_glob() {
  local app_name="$1"
  local extra_glob="$2"
  local case_dir="$3"
  [[ -z "$extra_glob" ]] && return 0
```

```
  local extra_dir="$case_dir/extra_artifacts"
  mkdir -p "$extra_dir"
  compgen -G "$extra_glob" > "$extra_dir/matched_paths.txt" || true
  if [[ -f "$extra_dir/matched_paths.txt" ]]; then
    while IFS= read -r p; do
      [[ -z "$p" ]] && continue
      local base out kind
      base="$(basename "$p")"
      out="$extra_dir/$(safe_name "$base")"
      kind="extra"
      if [[ -f "$p" && "$p" == *.plist ]]; then kind="plist"; fi
      if [[ -f "$p" && "$p" == *.dylib ]]; then kind="dylib"; fi
      if [[ -d "$p" && "$p" == *.app ]]; then kind="nested_app"; fi
      capture_path_artifact "$app_name" extra "$kind" "$p" "$out"
    done < "$extra_dir/matched_paths.txt"
  fi
}

capture_pcap_glob() {
  local app_name="$1"
  local pcap_glob="$2"
  local case_dir="$3"
  [[ -z "$pcap_glob" ]] && return 0
  local pcap_dir="$case_dir/pcaps"
  mkdir -p "$pcap_dir"
  compgen -G "$pcap_glob" > "$pcap_dir/matched_pcaps.txt" || true
```

```
  if [[ -f "$pcap_dir/matched_pcaps.txt" ]]; then
    while IFS= read -r p; do
      [[ -z "$p" ]] && continue
      local base out
      base="$(basename "$p")"
      out="$pcap_dir/$(safe_name "$base")"
      capture_path_artifact "$app_name" network "pcap" "$p" "$out"
    done < "$pcap_dir/matched_pcaps.txt"
  fi
}

capture_processes() {
  local app_name="$1"
  local match="$2"
  local case_dir="$3"
  local proc_dir="$case_dir/process"
  mkdir -p "$proc_dir"
  [[ -z "$match" ]] && match="$app_name"
  pgrep -fal "$match" > "$proc_dir/pgrep.txt" 2>&1 || true
  ps aux | grep -i "$match" > "$proc_dir/ps.txt" 2>&1 || true
}

capture_user_tcc() {
  local case_dir="$1"
  local tcc_dir="$case_dir/tcc"
```

```
  mkdir -p "$tcc_dir"

  local db="$HOME/Library/Application Support/com.apple.TCC/TCC.db"

  [[ -f "$db" ]] || return 0



  sqlite3 "$db" 'select
service,client,client_type,auth_value,auth_reason,indirect_object_identifier,last_modified from access
order by last_modified desc;' \

    > "$tcc_dir/user_tcc_access.txt" 2>/dev/null || true



  sort -u "$case_dir/bundle_ids.all" "$case_dir/code_identifiers.all" 2>/dev/null | sed '/^$/d' > "$tcc_dir/
targets.txt" || true



  : > "$tcc_dir/target_rows.txt"

  if [[ -s "$tcc_dir/targets.txt" && -f "$tcc_dir/user_tcc_access.txt" ]]; then

    while IFS= read -r target; do

      grep -F "$target" "$tcc_dir/user_tcc_access.txt" >> "$tcc_dir/target_rows.txt" || true

    done < "$tcc_dir/targets.txt"

  fi

}



capture_tccd_logs() {

  local app_name="$1"

  local match="$2"

  local case_dir="$3"

  local log_dir="$case_dir/tccd"

  mkdir -p "$log_dir"

  [[ -z "$match" ]] && match="$app_name"
```

```
  log show --last 20m --style compact --predicate 'process == "tccd"' > "$log_dir/tccd_last20m.txt" 2>/dev/
null || true

  log show --last 20m --style compact --predicate "eventMessage CONTAINS[c] '$match' OR
eventMessage CONTAINS[c] 'microphone' OR eventMessage CONTAINS[c] 'camera' OR eventMessage
CONTAINS[c] 'screen'" > "$log_dir/tccd_filtered.txt" 2>/dev/null || true

}



write_case_readme() {

  local app_name="$1"

  local case_dir="$2"

  cat > "$case_dir/README.txt" <<EOF

Case: $app_name

Generated: $(date -u +%Y-%m-%dT%H:%M:%SZ)



What this case contains:

- suspect app static verification

- baseline app static verification

- recursive inventory of app artifacts

- nested runtime discovery and trust checks

- frameworks / payload hashes and static inspection

- process snapshots if running

- user TCC rows for discovered bundle ids

- filtered tccd logs

- optional PCAP hashes



Use this case to answer only these questions:

1. Does the suspect outer launcher fail trust validation?
```

```
2. Does the fresh baseline outer launcher validate?

3. Does the nested runtime remain signed and operational?

4. Is there live helper/process activity consistent with the nested runtime?

5. Do TCC/tccd artifacts support privacy-relevant runtime behavior?

EOF

}


write_case_md_summary() {

  local app_name="$1"

  local case_dir="$2"

  local md="$case_dir/EXEC_SUMMARY.md"

  cat > "$md" <<EOF

# $app_name Case Executive Summary


## Reviewer workflow

1. Open \

   - summary row in ../../summary.tsv

   - artifact inventory in ../../artifact_summary.tsv

2. Review suspect static outputs under \

   - suspect/

3. Review baseline static outputs under \

   - baseline/

4. Review recursive bundle inventory under \

   - suspect_inventory/

   - baseline_inventory/

5. Review process/TCC/tccd support under \
```

```
   - process/
   - tcc/
   - tccd/

## Pass/fail questions
- suspect outer launcher fails trust validation?
- baseline outer launcher passes trust validation?
- nested runtime remains signed?
- helper/process tree is visible during runtime?
- privacy-relevant runtime artifacts are present?

## Notes
This packet is scoped to app-integrity, nested-runtime, process, and TCC/tccd support only.
EOF
}

log "Run directory: $RUN_DIR"
cp "$MANIFEST" "$RUN_DIR/manifest.csv"

{ read -r _header || true; while IFS= read -r line || [[ -n "$line" ]]; do
  [[ -z "$line" ]] && continue
  name="$(trim "$(csv_get "$line" 1)")"
  suspect_app="$(trim "$(csv_get "$line" 2)")"
  baseline_app="$(trim "$(csv_get "$line" 3)")"
  process_match="$(trim "$(csv_get "$line" 4)")"
```

```
pcap_glob="$(trim "$(csv_get "$line" 5)")"
extra_glob="$(trim "$(csv_get "$line" 6)")"

[[ -z "$name" ]] && continue

case_dir="$RUN_DIR/$(safe_name "$name")"
mkdir -p "$case_dir"
: > "$case_dir/bundle_ids.all"
: > "$case_dir/code_identifiers.all"

log "=== Capturing case: $name ==="
printf '%s\n' "$name" > "$case_dir/app_name.txt"
printf '%s\n' "$suspect_app" > "$case_dir/suspect_path.txt"
printf '%s\n' "$baseline_app" > "$case_dir/baseline_path.txt"
printf '%s\n' "$process_match" > "$case_dir/process_match.txt"

capture_static_bundle "$name" suspect "$suspect_app" "$case_dir"
recursive_inventory_bundle "$name" suspect "$suspect_app" "$case_dir"
capture_framework_payload_strings "$suspect_app" "$case_dir/suspect_inventory"
capture_nested_runtimes "$name" suspect "$suspect_app" "$case_dir"

capture_static_bundle "$name" baseline "$baseline_app" "$case_dir"
recursive_inventory_bundle "$name" baseline "$baseline_app" "$case_dir"
capture_framework_payload_strings "$baseline_app" "$case_dir/baseline_inventory"
capture_nested_runtimes "$name" baseline "$baseline_app" "$case_dir"
```

```
  capture_extra_glob "$name" "$extra_glob" "$case_dir"
  capture_pcap_glob "$name" "$pcap_glob" "$case_dir"
  capture_processes "$name" "$process_match" "$case_dir"
  capture_user_tcc "$case_dir"
  capture_tccd_logs "$name" "$process_match" "$case_dir"
  write_case_readme "$name" "$case_dir"
  write_case_md_summary "$name" "$case_dir"

done; } < "$MANIFEST"

cut -f1-11 "$SUMMARY" > "$RUN_DIR/summary_core.tsv" 2>/dev/null || true

cat > "$RUN_DIR/reviewer_run_order.txt" <<'EOF'
Reviewer order:
1. Open summary.tsv
2. Compare suspect vs baseline rows for one app
3. Read suspect/codesign_bundle.txt and suspect/spctl_bundle.txt
4. Read baseline/codesign_bundle.txt and baseline/spctl_bundle.txt
5. Read suspect_nested_*/codesign_executable.txt if present
6. Read process/pgrep.txt and process/ps.txt
7. Read tcc/target_rows.txt and tccd/tccd_filtered.txt
8. Use artifact_summary.tsv to inspect every recursively captured plist, executable, dylib, framework, xpc, helper app, and provisioning profile
EOF

log "Done. Results at: $RUN_DIR"
```

```
echo "$RUN_DIR"#!/bin/bash

set -euo pipefail


# atlas_submission_capture.sh

#

# Purpose:

#   Build a short, repeatable, reviewer-friendly evidence packet for Atlas and

#   any clone-app comparisons using the same workflow.

#

# Key requirement:

#   This script DOES recurse through app directories. It does not stop at the

#   head of a directory. It hunts through the bundle for:

#     - executables

#     - nested .app bundles

#     - Info.plist and other plist files

#     - dylibs / frameworks / xpc services

#     - helper apps

#     - provisioning profiles

#     - launchd-related plists inside the bundle

#

# What it captures per app pair:

#   - suspect outer launcher metadata, hashes, trust results

#   - baseline outer launcher metadata, hashes, trust results

#   - recursive inventory of important bundle artifacts

#   - nested runtime discovery + validation
```

```
#   - interesting Frameworks payload hashes (e.g. OwlBridge.dylib)
#   - live process captures if the app is running
#   - user TCC rows for discovered bundle ids
#   - filtered tccd logs for recent runtime activity
#   - optional PCAP hashes if you place captures next to the app case
#   - per-app markdown executive summary
#
# What it does NOT do:
#   - no sudo
#   - no writes to app bundles
#   - no quarantine stripping
#   - no execution from suspect bundles
#   - no Microsoft / Google / MDM / Recovery scope creep
#
# ------------------------------------------------------------------------
# MANIFEST FORMAT
# ------------------------------------------------------------------------
# Provide a CSV with this header:
# name,suspect_app,baseline_app,process_match,pcap_glob,extra_glob
#
# Example:
# name,suspect_app,baseline_app,process_match,pcap_glob,extra_glob
# Atlas,/Applications/ChatGPT Atlas.app,/Applications/ChatGPT Atlas 2.app,"ChatGPT Atlas","/Volumes/Ellis/Investigations/pcaps/*ATLAS*.cap","/Volumes/Ellis/Investigations/atlas_helpers/*"
# Codex,/Applications/Codex.app,/Applications/Codex 2.app,Codex,,
# Chrome,/Applications/Google Chrome.app,/Applications/Google Chrome Fresh.app,"Google Chrome",,
```

```
#
# Run:
#   chmod +x atlas_submission_capture.sh
#   ./atlas_submission_capture.sh manifest.csv /Volumes/Ellis/Investigations
#
# Results:
#   /Volumes/Ellis/Investigations/atlas_submission_capture_<UTC>/...
# ------------------------------------------------------------------------

if [[ $# -lt 1 || $# -gt 2 ]]; then
  echo "Usage: $0 <manifest.csv> [output_base]"
  exit 1
fi

MANIFEST="$1"
OUT_BASE="${2:-$PWD}"

if [[ ! -f "$MANIFEST" ]]; then
  echo "[FATAL] Manifest not found: $MANIFEST"
  exit 1
fi

mkdir -p "$OUT_BASE"
OUT_BASE="$(cd "$OUT_BASE" && pwd -P)"
RUN_TS="$(date -u +%Y%m%dT%H%M%SZ)"
RUN_DIR="$OUT_BASE/atlas_submission_capture_${RUN_TS}"
```

```
mkdir -p "$RUN_DIR"



LOG="$RUN_DIR/run.log"

SUMMARY="$RUN_DIR/summary.tsv"

ARTIFACT_SUMMARY="$RUN_DIR/artifact_summary.tsv"

printf
'app\trole\tpath\tsha256\tsize\tbirth\tmtime\tbundle_id\tteam_id\tcodesign_status\tspctl_status\tnotes\n' >
"$SUMMARY"

printf 'app\trole\tartifact_type\tpath\tsha256\tsize\tbirth\tmtime\tbundle_id\tteam_id\tnotes\n' >
"$ARTIFACT_SUMMARY"



log() {

  printf '[%s] %s\n' "$(date -u +%Y-%m-%dT%H:%M:%SZ)" "$*" | tee -a "$LOG"

}



trim() {

  local s="$1"

  s="${s#${s%%[![:space:]]*}}"

  s="${s%${s##*[![:space:]]}}"

  printf '%s' "$s"

}



csv_get() {

  local line="$1" idx="$2"

  awk -F',' -v i="$idx" '{gsub(/^"|"$/, "", $i); print $i}' <<< "$line"

}
```

```
safe_name() {
  printf '%s' "$1" | tr -cs 'A-Za-z0-9._-' '_'
}

sha256_file() {
  local f="$1"
  shasum -a 256 "$f" | awk '{print $1}'
}

file_size() {
  stat -f '%z' "$1" 2>/dev/null || echo ""
}

birth_time() {
  stat -f '%SB' -t '%Y-%m-%dT%H:%M:%SZ' "$1" 2>/dev/null || echo ""
}

mtime_time() {
  stat -f '%Sm' -t '%Y-%m-%dT%H:%M:%SZ' "$1" 2>/dev/null || echo ""
}

bundle_id_of_app() {
  local app="$1"
  /usr/libexec/PlistBuddy -c 'Print :CFBundleIdentifier' "$app/Contents/Info.plist" 2>/dev/null || true
}
```

```
bundle_executable_of_app() {
  local app="$1"
  /usr/libexec/PlistBuddy -c 'Print :CFBundleExecutable' "$app/Contents/Info.plist" 2>/dev/null || true
}

capture_codesign() {
  local target="$1" out="$2"
  if codesign -dv --verbose=4 "$target" > "$out" 2>&1; then
    echo "ok"
  else
    echo "fail"
  fi
}

capture_spctl() {
  local target="$1" out="$2"
  if spctl -a -vv "$target" > "$out" 2>&1; then
    echo "accepted"
  else
    echo "rejected"
  fi
}

extract_team_id() {
  local f="$1"
```

```
  awk -F= '/^TeamIdentifier=/{print $2; exit}' "$f" | tr -d '[:space:]'
}

extract_identifier() {
  local f="$1"
  awk -F= '/^Identifier=/{print $2; exit}' "$f" | tr -d '[:space:]'
}

is_macho_executable() {
  local f="$1"
  file "$f" 2>/dev/null | grep -Eq 'Mach-O.*(executable|dynamically linked shared library|bundle)'
}

add_artifact_summary() {
  local app_name="$1" role="$2" artifact_type="$3" path="$4" sha="$5" size="$6" birth="$7" mtime="$8" bundle_id="$9" team_id="${10}" notes="${11}"
  printf '%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\n' \
    "$app_name" "$role" "$artifact_type" "$path" "$sha" "$size" "$birth" "$mtime" "$bundle_id" "$team_id" "$notes" >> "$ARTIFACT_SUMMARY"
}

capture_path_artifact() {
  local app_name="$1" role="$2" artifact_type="$3" path="$4" out_dir="$5"
  mkdir -p "$out_dir"
  printf '%s\n' "$path" > "$out_dir/path.txt"
```

```
local sha="" size="" birth="" mtime="" bundle_id="" team_id="" notes=""
if [[ -f "$path" ]]; then
  sha="$(sha256_file "$path" 2>/dev/null || true)"
  size="$(file_size "$path")"
  birth="$(birth_time "$path")"
  mtime="$(mtime_time "$path")"
  file "$path" > "$out_dir/file.txt" 2>&1 || true
  stat "$path" > "$out_dir/stat.txt" 2>&1 || true
  xattr -lr "$path" > "$out_dir/xattr.txt" 2>&1 || true
  shasum -a 256 "$path" > "$out_dir/sha256.txt" 2>&1 || true

  if is_macho_executable "$path"; then
    otool -L "$path" > "$out_dir/otool_L.txt" 2>&1 || true
    otool -l "$path" > "$out_dir/otool_l.txt" 2>&1 || true
    codesign -dv --verbose=4 "$path" > "$out_dir/codesign.txt" 2>&1 || true
    bundle_id="$(extract_identifier "$out_dir/codesign.txt")"
    team_id="$(extract_team_id "$out_dir/codesign.txt")"
  fi

  if [[ "$artifact_type" == "plist" || "$path" == *.plist ]]; then
    plutil -p "$path" > "$out_dir/plutil.txt" 2>&1 || true
  fi
elif [[ -d "$path" ]]; then
  stat "$path" > "$out_dir/stat.txt" 2>&1 || true
  xattr -lr "$path" > "$out_dir/xattr.txt" 2>&1 || true
  notes="directory"
```

```
  else
    notes="missing"
  fi

  add_artifact_summary "$app_name" "$role" "$artifact_type" "$path" "$sha" "$size" "$birth" "$mtime"
"$bundle_id" "$team_id" "$notes"
}

capture_static_bundle() {
  local app_name="$1"
  local role="$2"
  local app_path="$3"
  local case_dir="$4"

  if [[ ! -d "$app_path" ]]; then
    log "[WARN] Missing $role app for $app_name: $app_path"
    return 0
  fi

  local role_dir="$case_dir/${role}"
  mkdir -p "$role_dir"

  local exec_name exec_path sha size birth mtime bundle_id codesign_state spctl_state team_id ident
  exec_name="$(bundle_executable_of_app "$app_path")"
  bundle_id="$(bundle_id_of_app "$app_path")"
  exec_path="$app_path/Contents/MacOS/$exec_name"
```

```
printf '%s\n' "$app_path" > "$role_dir/app_path.txt"
printf '%s\n' "$bundle_id" > "$role_dir/bundle_id.txt"
printf '%s\n' "$exec_name" > "$role_dir/executable_name.txt"

if [[ -f "$exec_path" ]]; then
  sha="$(sha256_file "$exec_path")"
  size="$(file_size "$exec_path")"
  birth="$(birth_time "$exec_path")"
  mtime="$(mtime_time "$exec_path")"
  file "$exec_path" > "$role_dir/file.txt" 2>&1 || true
  stat "$exec_path" > "$role_dir/stat.txt" 2>&1 || true
  xattr -lr "$exec_path" > "$role_dir/xattr.txt" 2>&1 || true
  otool -L "$exec_path" > "$role_dir/otool_L.txt" 2>&1 || true
  otool -l "$exec_path" > "$role_dir/otool_l.txt" 2>&1 || true
  shasum -a 256 "$exec_path" > "$role_dir/sha256.txt" 2>&1 || true
else
  sha=""
  size=""
  birth=""
  mtime=""
  printf '[WARN] Missing executable: %s\n' "$exec_path" > "$role_dir/file.txt"
fi

codesign_state="$(capture_codesign "$app_path" "$role_dir/codesign_bundle.txt")"
```

```
  spctl_state="$(capture_spctl "$app_path" "$role_dir/spctl_bundle.txt")"

  if [[ -f "$exec_path" ]]; then
    capture_codesign "$exec_path" "$role_dir/codesign_executable.txt" >/dev/null ll true
  fi

  ident="$(extract_identifier "$role_dir/codesign_executable.txt")"
  team_id="$(extract_team_id "$role_dir/codesign_executable.txt")"

  printf '%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\n' \
    "$app_name" "$role" "$app_path" "$sha" "$size" "$birth" "$mtime" "$bundle_id" "$team_id"
"$codesign_state" "$spctl_state" "$ident" >> "$SUMMARY"

  printf '%s\n' "$bundle_id" >> "$case_dir/bundle_ids.all"
  printf '%s\n' "$ident" >> "$case_dir/code_identifiers.all"

  add_artifact_summary "$app_name" "$role" "app_bundle" "$app_path" "" "" "" "" "$bundle_id" "$team_id"
"$ident"
  if [[ -f "$exec_path" ]]; then
    add_artifact_summary "$app_name" "$role" "outer_executable" "$exec_path" "$sha" "$size" "$birth"
"$mtime" "$bundle_id" "$team_id" "$ident"
  fi
}

recursive_inventory_bundle() {
  local app_name="$1"
  local role="$2"
```

```
local app_path="$3"
local case_dir="$4"
local inv_dir="$case_dir/${role}_inventory"
mkdir -p "$inv_dir"

local index=0
while IFS= read -r p; do
  [[ -z "$p" ]] && continue
  index=$((index+1))
  local base stem out artifact_type
  base="$(basename "$p")"
  stem="$(printf '%05d_%s' "$index" "$(safe_name "$base")")"
  out="$inv_dir/$stem"

  artifact_type="file"
  if [[ -d "$p" && "$p" == *.app ]]; then artifact_type="nested_app"; fi
  if [[ -d "$p" && "$p" == *.xpc ]]; then artifact_type="xpc_service"; fi
  if [[ -f "$p" && "$p" == *.plist ]]; then artifact_type="plist"; fi
  if [[ -f "$p" && "$p" == *.dylib ]]; then artifact_type="dylib"; fi
  if [[ -d "$p" && "$p" == *.framework ]]; then artifact_type="framework"; fi
  if [[ -f "$p" && "$p" == *embedded.provisionprofile ]]; then artifact_type="provisioning_profile"; fi

  capture_path_artifact "$app_name" "$role" "$artifact_type" "$p" "$out"
done < <(
  find "$app_path" \(
    -type d \( -name '*.app' -o -name '*.framework' -o -name '*.xpc' \) -o \
```

```
      -type f \( -name '*.plist' -o -name '*.dylib' -o -name 'embedded.provisionprofile' -o -perm -111 \)
    \) | sort
  )
}

capture_framework_payload_strings() {
  local app_path="$1"
  local inv_dir="$2"
  [[ -d "$inv_dir" ]] || return 0
  while IFS= read -r file; do
    [[ -z "$file" ]] && continue
    if [[ -f "$file/path.txt" ]]; then
      local p
      p="$(cat "$file/path.txt")"
      if [[ -f "$p" && ( "$p" == *.dylib || "$p" == *OwlBridge* || "$p" == *Bridge* ) ]]; then
        strings "$p" | grep -Ei 'owlbridge|icloud|keychain|controller|dyld|bridge' > "$file/interesting_strings.txt"
2>/dev/null || true
      fi
    fi
  done < <(find "$inv_dir" -type d | sort)
}

capture_nested_runtimes() {
  local app_name="$1"
  local role="$2"
  local app_path="$3"
```

```
  local case_dir="$4"
  local nested_dir="$case_dir/${role}_nested"
  mkdir -p "$nested_dir"

  local nested index=0
  while IFS= read -r nested; do
    [[ -z "$nested" ]] && continue
    [[ "$nested" == "$app_path" ]] && continue
    index=$((index+1))
    capture_static_bundle "$app_name" "${role}_nested_${index}" "$nested" "$case_dir"
  done < <(find "$app_path" -type d -name '*.app' | sort)
}

capture_extra_glob() {
  local app_name="$1"
  local extra_glob="$2"
  local case_dir="$3"
  [[ -z "$extra_glob" ]] && return 0
  local extra_dir="$case_dir/extra_artifacts"
  mkdir -p "$extra_dir"
  compgen -G "$extra_glob" > "$extra_dir/matched_paths.txt" || true
  if [[ -f "$extra_dir/matched_paths.txt" ]]; then
    while IFS= read -r p; do
      [[ -z "$p" ]] && continue
      local base out kind
```

```
      base="$(basename "$p")"
      out="$extra_dir/$(safe_name "$base")"
      kind="extra"
      if [[ -f "$p" && "$p" == *.plist ]]; then kind="plist"; fi
      if [[ -f "$p" && "$p" == *.dylib ]]; then kind="dylib"; fi
      if [[ -d "$p" && "$p" == *.app ]]; then kind="nested_app"; fi
      capture_path_artifact "$app_name" extra "$kind" "$p" "$out"
    done < "$extra_dir/matched_paths.txt"
  fi
}

capture_pcap_glob() {
  local app_name="$1"
  local pcap_glob="$2"
  local case_dir="$3"
  [[ -z "$pcap_glob" ]] && return 0
  local pcap_dir="$case_dir/pcaps"
  mkdir -p "$pcap_dir"
  compgen -G "$pcap_glob" > "$pcap_dir/matched_pcaps.txt" || true
  if [[ -f "$pcap_dir/matched_pcaps.txt" ]]; then
    while IFS= read -r p; do
      [[ -z "$p" ]] && continue
      local base out
      base="$(basename "$p")"
      out="$pcap_dir/$(safe_name "$base")"
      capture_path_artifact "$app_name" network "pcap" "$p" "$out"
```

```
    done < "$pcap_dir/matched_pcaps.txt"
  fi
}

capture_processes() {
  local app_name="$1"
  local match="$2"
  local case_dir="$3"
  local proc_dir="$case_dir/process"
  mkdir -p "$proc_dir"
  [[ -z "$match" ]] && match="$app_name"
  pgrep -fal "$match" > "$proc_dir/pgrep.txt" 2>&1 || true
  ps aux | grep -i "$match" > "$proc_dir/ps.txt" 2>&1 || true
}

capture_user_tcc() {
  local case_dir="$1"
  local tcc_dir="$case_dir/tcc"
  mkdir -p "$tcc_dir"
  local db="$HOME/Library/Application Support/com.apple.TCC/TCC.db"
  [[ -f "$db" ]] || return 0

  sqlite3 "$db" 'select
service,client,client_type,auth_value,auth_reason,indirect_object_identifier,last_modified from access
order by last_modified desc;' \
    > "$tcc_dir/user_tcc_access.txt" 2>/dev/null || true
```

```
  sort -u "$case_dir/bundle_ids.all" "$case_dir/code_identifiers.all" 2>/dev/null | sed '/^$/d' > "$tcc_dir/
targets.txt" || true



  : > "$tcc_dir/target_rows.txt"

  if [[ -s "$tcc_dir/targets.txt" && -f "$tcc_dir/user_tcc_access.txt" ]]; then

    while IFS= read -r target; do

      grep -F "$target" "$tcc_dir/user_tcc_access.txt" >> "$tcc_dir/target_rows.txt" || true

    done < "$tcc_dir/targets.txt"

  fi

}



capture_tccd_logs() {

  local app_name="$1"

  local match="$2"

  local case_dir="$3"

  local log_dir="$case_dir/tccd"

  mkdir -p "$log_dir"

  [[ -z "$match" ]] && match="$app_name"

  log show --last 20m --style compact --predicate 'process == "tccd"' > "$log_dir/tccd_last20m.txt" 2>/dev/
null || true

  log show --last 20m --style compact --predicate "eventMessage CONTAINS[c] '$match' OR
eventMessage CONTAINS[c] 'microphone' OR eventMessage CONTAINS[c] 'camera' OR eventMessage
CONTAINS[c] 'screen'" > "$log_dir/tccd_filtered.txt" 2>/dev/null || true

}



write_case_readme() {

  local app_name="$1"
```

```
  local case_dir="$2"
  cat > "$case_dir/README.txt" <<EOF
Case: $app_name
Generated: $(date -u +%Y-%m-%dT%H:%M:%SZ)

What this case contains:
- suspect app static verification
- baseline app static verification
- recursive inventory of app artifacts
- nested runtime discovery and trust checks
- frameworks / payload hashes and static inspection
- process snapshots if running
- user TCC rows for discovered bundle ids
- filtered tccd logs
- optional PCAP hashes

Use this case to answer only these questions:
1. Does the suspect outer launcher fail trust validation?
2. Does the fresh baseline outer launcher validate?
3. Does the nested runtime remain signed and operational?
4. Is there live helper/process activity consistent with the nested runtime?
5. Do TCC/tccd artifacts support privacy-relevant runtime behavior?
EOF
}
```

```
write_case_md_summary() {
  local app_name="$1"
  local case_dir="$2"
  local md="$case_dir/EXEC_SUMMARY.md"
  cat > "$md" <<EOF
# $app_name Case Executive Summary

## Reviewer workflow
1. Open \
   - summary row in ../../summary.tsv
   - artifact inventory in ../../artifact_summary.tsv
2. Review suspect static outputs under \
   - suspect/
3. Review baseline static outputs under \
   - baseline/
4. Review recursive bundle inventory under \
   - suspect_inventory/
   - baseline_inventory/
5. Review process/TCC/tccd support under \
   - process/
   - tcc/
   - tccd/

## Pass/fail questions
- suspect outer launcher fails trust validation?
- baseline outer launcher passes trust validation?
```

- nested runtime remains signed?

- helper/process tree is visible during runtime?

- privacy-relevant runtime artifacts are present?

Notes

This packet is scoped to app-integrity, nested-runtime, process, and TCC/tccd support only.

EOF

}

log "Run directory: $RUN_DIR"

cp "$MANIFEST" "$RUN_DIR/manifest.csv"

{ read -r _header || true; while IFS= read -r line || [[-n "$line"]]; do

  [[-z "$line"]] && continue

  name="$(trim "$(csv_get "$line" 1)")"

  suspect_app="$(trim "$(csv_get "$line" 2)")"

  baseline_app="$(trim "$(csv_get "$line" 3)")"

  process_match="$(trim "$(csv_get "$line" 4)")"

  pcap_glob="$(trim "$(csv_get "$line" 5)")"

  extra_glob="$(trim "$(csv_get "$line" 6)")"

  [[-z "$name"]] && continue

  case_dir="$RUN_DIR/$(safe_name "$name")"

  mkdir -p "$case_dir"

```
: > "$case_dir/bundle_ids.all"
: > "$case_dir/code_identifiers.all"

log "=== Capturing case: $name ==="
printf '%s\n' "$name" > "$case_dir/app_name.txt"
printf '%s\n' "$suspect_app" > "$case_dir/suspect_path.txt"
printf '%s\n' "$baseline_app" > "$case_dir/baseline_path.txt"
printf '%s\n' "$process_match" > "$case_dir/process_match.txt"

capture_static_bundle "$name" suspect "$suspect_app" "$case_dir"
recursive_inventory_bundle "$name" suspect "$suspect_app" "$case_dir"
capture_framework_payload_strings "$suspect_app" "$case_dir/suspect_inventory"
capture_nested_runtimes "$name" suspect "$suspect_app" "$case_dir"

capture_static_bundle "$name" baseline "$baseline_app" "$case_dir"
recursive_inventory_bundle "$name" baseline "$baseline_app" "$case_dir"
capture_framework_payload_strings "$baseline_app" "$case_dir/baseline_inventory"
capture_nested_runtimes "$name" baseline "$baseline_app" "$case_dir"

capture_extra_glob "$name" "$extra_glob" "$case_dir"
capture_pcap_glob "$name" "$pcap_glob" "$case_dir"
capture_processes "$name" "$process_match" "$case_dir"
capture_user_tcc "$case_dir"
capture_tccd_logs "$name" "$process_match" "$case_dir"
write_case_readme "$name" "$case_dir"
write_case_md_summary "$name" "$case_dir"
```

```
done; } < "$MANIFEST"

cut -f1-11 "$SUMMARY" > "$RUN_DIR/summary_core.tsv" 2>/dev/null || true

cat > "$RUN_DIR/reviewer_run_order.txt" <<'EOF'
Reviewer order:
1. Open summary.tsv
2. Compare suspect vs baseline rows for one app
3. Read suspect/codesign_bundle.txt and suspect/spctl_bundle.txt
4. Read baseline/codesign_bundle.txt and baseline/spctl_bundle.txt
5. Read suspect_nested_*/codesign_executable.txt if present
6. Read process/pgrep.txt and process/ps.txt
7. Read tcc/target_rows.txt and tccd/tccd_filtered.txt
8. Use artifact_summary.tsv to inspect every recursively captured plist, executable, dylib, framework, xpc, helper app, and provisioning profile
EOF

log "Done. Results at: $RUN_DIR"
echo "$RUN_DIR"

#!/bin/bash
# Read-only, per-object verification for mounted macOS evidence volumes.
#
# Default:
#   /Volumes/Ellis/scripts/os_boot_verify_v2.sh /Volumes/OS_BOOT /Volumes/RESCUE_OS
```

```
#
# This script never executes code from the evidence volume. It inventories every
# object, hashes and statically checks code-like files individually, validates
# app/package/disk-image containers, and writes all results outside the source.

set -u
set -o pipefail

PATH="/usr/bin:/bin:/usr/sbin:/sbin:$HOME/.homebrew/bin:$PATH"
export PATH LC_ALL=C LANG=C

SCRIPT_HOME="$(cd -- "$(dirname -- "$0")" && pwd -P)"
OUT_BASE="$SCRIPT_HOME/results"
ALLOW_WRITABLE=0
HASH_MODE="code"
MAX_TEXT_MB=16
LIMIT_FILES=0
INSPECT_DMG_CONTENTS=0
DMG_MAX_FILES=100000
CONTAINER_ONLY=""
VOLUMES=()
ATTACHED_DEVICES=()

usage() {
  cat <<'EOF'
```

```
Usage: os_boot_verify_v2.sh [options] [VOLUME ...]

Options:
  --out-base DIR       Result parent directory (default: script/results)
  --hash-all           SHA-256 every regular file, including non-code data
  --no-hash            Do not hash regular files
  --max-text-mb N      Maximum text-file size searched by ripgrep (default: 16)
  --limit-files N      Stop after N regular files per volume; marks run partial
  --inspect-dmg-contents
                       Mount DMGs read-only, inventory internal objects, and
                       verify nested apps/containers without executing content
  --dmg-max-files N    Maximum internal objects inventoried per DMG (default: 100000)
  --container-only FILE
                       Verify one package/archive/DMG without walking its volume
  --allow-writable     Permit a writable source (intended only for test fixtures)
  -h, --help           Show this help

With no VOLUME arguments, mounted /Volumes/OS_BOOT and /Volumes/RESCUE_OS are used.
EOF
}

fatal() {
  echo "[FATAL] $*" >&2
  exit 2
}
```

```
while [ "$#" -gt 0 ]; do
  case "$1" in
    --out-base)
      [ "$#" -ge 2 ] || fatal "--out-base requires a directory"
      OUT_BASE="$2"
      shift 2
      ;;
    --hash-all)
      HASH_MODE="all"
      shift
      ;;
    --no-hash)
      HASH_MODE="none"
      shift
      ;;
    --max-text-mb)
      [ "$#" -ge 2 ] || fatal "--max-text-mb requires a number"
      MAX_TEXT_MB="$2"
      shift 2
      ;;
    --limit-files)
      [ "$#" -ge 2 ] || fatal "--limit-files requires a number"
      LIMIT_FILES="$2"
      shift 2
      ;;
```

```
--inspect-dmg-contents)
  INSPECT_DMG_CONTENTS=1
  shift
  ;;
--dmg-max-files)
  [ "$#" -ge 2 ] || fatal "--dmg-max-files requires a number"
  DMG_MAX_FILES="$2"
  shift 2
  ;;
--container-only)
  [ "$#" -ge 2 ] || fatal "--container-only requires a file"
  CONTAINER_ONLY="$2"
  shift 2
  ;;
--allow-writable)
  ALLOW_WRITABLE=1
  shift
  ;;
-h|--help)
  usage
  exit 0
  ;;
--)
  shift
  while [ "$#" -gt 0 ]; do
    VOLUMES[${#VOLUMES[@]}]="$1"
```

```
        shift
      done
      ;;
    -*)
      fatal "unknown option: $1"
      ;;
    *)
      VOLUMES[${#VOLUMES[@]}]="$1"
      shift
      ;;
  esac
done

case "$MAX_TEXT_MB" in *[!0-9]*|'') fatal "--max-text-mb must be an integer";; esac
case "$LIMIT_FILES" in *[!0-9]*|'') fatal "--limit-files must be an integer";; esac
case "$DMG_MAX_FILES" in *[!0-9]*|'') fatal "--dmg-max-files must be an integer";; esac

if [ "${#VOLUMES[@]}" -eq 0 ] && [ -z "$CONTAINER_ONLY" ]; then
  [ -d /Volumes/OS_BOOT ] && VOLUMES[${#VOLUMES[@]}]="/Volumes/OS_BOOT"
  [ -d /Volumes/RESCUE_OS ] && VOLUMES[${#VOLUMES[@]}]="/Volumes/RESCUE_OS"
fi
if [ -z "$CONTAINER_ONLY" ]; then
  [ "${#VOLUMES[@]}" -gt 0 ] || fatal "no source volumes were supplied or mounted"
else
  [ -f "$CONTAINER_ONLY" ] || fatal "container is not a regular file: $CONTAINER_ONLY"
```

```
  CONTAINER_ONLY="$(cd -- "$(dirname -- "$CONTAINER_ONLY")" && pwd -P)/$(basename --
"$CONTAINER_ONLY")"
fi

mkdir -p "$OUT_BASE" || fatal "cannot create output base: $OUT_BASE"
OUT_BASE="$(cd "$OUT_BASE" && pwd -P)"
UTC_STAMP="$(date -u +%Y%m%dT%H%M%SZ)"
CASE="USB_verify_v2_${UTC_STAMP}"
OUT="$OUT_BASE/$CASE"
mkdir -p "$OUT"
touch "$OUT/INCOMPLETE"

RUN_COMPLETE=0
on_exit() {
  rc=$?
  for attached_device in "${ATTACHED_DEVICES[@]-}"; do
    [ -n "$attached_device" ] || continue
    hdiutil detach "$attached_device" >/dev/null 2>&1 || true
  done
  if [ "$RUN_COMPLETE" -eq 1 ]; then
    rm -f "$OUT/INCOMPLETE"
    touch "$OUT/COMPLETE"
  else
    date -u +%Y-%m-%dT%H:%M:%SZ > "$OUT/00_utc_end.txt" 2>/dev/null || true
    printf 'exit_code=%s\n' "$rc" > "$OUT/INTERRUPTED.txt" 2>/dev/null || true
  fi
```

```
  trap - EXIT
  exit "$rc"
}

mark_detached() {
  detached_device=$1
  for attached_index in "${!ATTACHED_DEVICES[@]}"; do
    if [ "${ATTACHED_DEVICES[$attached_index]}" = "$detached_device" ]; then
      unset 'ATTACHED_DEVICES[attached_index]'
    fi
  done
}
trap on_exit EXIT
trap 'exit 130' INT
trap 'exit 143' TERM

LOG="$OUT/command_log.md"
ERRORS="$OUT/errors.tsv"
printf 'stage\tpath\texit_code\tmessage\n' > "$ERRORS"

log() {
  now="$(date -u +%Y-%m-%dT%H:%M:%SZ)"
  printf '[%s] %s\n' "$now" "$*" | tee -a "$LOG"
}

escape_tsv() {
```

```
  value=$1
  value=${value//\\/\\\\}
  value=${value//$'\t'/\\t}
  value=${value//$'\r'/\\r}
  value=${value//$'\n'/\\n}
  printf '%s' "$value"
}

record_error() {
  stage=$(escape_tsv "$1")
  path=$(escape_tsv "$2")
  code=$(escape_tsv "$3")
  message=$(escape_tsv "$4")
  printf '%s\t%s\t%s\t%s\n' "$stage" "$path" "$code" "$message" >> "$ERRORS"
}

safe_component() {
  printf '%s' "$1" | tr -cs 'A-Za-z0-9._-' '_' | cut -c1-80
}

path_id() {
  printf '%s' "$1" | shasum -a 256 | awk '{print substr($1,1,16)}'
}

relative_path() {
```

```
  path=$1
  root=$2
  if [ "$path" = "$root" ]; then
    printf '.'
  else
    printf '%s' "${path#"$root"/}"
  fi
}

top_name() {
  rel=$1
  case "$rel" in
    */*) printf '%s' "${rel%%/*}" ;;
    .) printf '_ROOT_' ;;
    *) printf '_ROOT_FILES' ;;
  esac
}

top_output_dir() {
  top=$1
  slug="$(safe_component "$top")_$(path_id "$top")"
  dir="$CURRENT_VOUT/directories/$slug"
  mkdir -p "$dir/details"
  if [ ! -e "$dir/objects.tsv" ]; then
    printf
'relative_path\tkind\tmode\tuid\tgid\tsize\tmtime_epoch\tbirth_epoch\tflags\tsha256\tclass\tfile_description\tl
ink_target\n' > "$dir/objects.tsv"
```

```
  fi

  if [ ! -e "$dir/code_verification.tsv" ]; then

    printf
'relative_path\tclass\tsha256\tstatic_parse\tcodesign\tgatekeeper\tidentifier\tteam_identifier\tauthorities\txat
tr_names\tquarantine\tdetail_file\n' > "$dir/code_verification.tsv"

  fi

  printf '%s' "$dir"

}



write_object_row() {

  out_file=$1

  rel=$2

  kind=$3

  mode=$4

  uid=$5

  gid=$6

  size=$7

  mtime=$8

  birth=$9

  flags=${10}

  sha=${11}

  class=${12}

  description=${13}

  link_target=${14}

  printf '%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\n' \

    "$(escape_tsv "$rel")" "$(escape_tsv "$kind")" "$(escape_tsv "$mode")" \
```

```
    "$(escape_tsv "$uid")" "$(escape_tsv "$gid")" "$(escape_tsv "$size")" \
    "$(escape_tsv "$mtime")" "$(escape_tsv "$birth")" "$(escape_tsv "$flags")" \
    "$(escape_tsv "$sha")" "$(escape_tsv "$class")" "$(escape_tsv "$description")" \
    "$(escape_tsv "$link_target")" >> "$out_file"
}

classify_file() {
  path=$1
  description=$2
  lower=$(printf '%s' "$path" | tr '[:upper:]' '[:lower:]')
  case "$description" in
    *Mach-O*) printf 'mach-o'; return ;;
  esac
  case "$lower" in
    *.dylibl*.sol*.bundle) printf 'native-library'; return ;;
    *.shl*.bashl*.zshl*.command) printf 'shell'; return ;;
    *.pyl*.pyw) printf 'python'; return ;;
    *.jsl*.mjsl*.cjs) printf 'javascript'; return ;;
    *.tsl*.tsx) printf 'typescript'; return ;;
    *.json) printf 'json'; return ;;
    *.plist) printf 'plist'; return ;;
    *.pkg) printf 'package'; return ;;
    *.dmg) printf 'disk-image'; return ;;
    *.zipl*.tarl*.tgzl*.tar.gzl*.tbzl*.tbz2l*.tar.bz2l*.tar.xz) printf 'archive'; return ;;
    *.peml*.keyl*.p12l*.pfx) printf 'key-material'; return ;;
  esac
```

```
  if [ -x "$path" ]; then
    if [ "$(head -c 2 "$path" 2>/dev/null)" = '#!' ]; then
      first_line=$(head -n 1 "$path" 2>/dev/null || true)
      case "$first_line" in
        *python*) printf 'python'; return ;;
        *node*) printf 'javascript'; return ;;
        *bash*|*zsh*|*'/sh'*) printf 'shell'; return ;;
      esac
    fi
    printf 'executable-other'
    return
  fi
  printf 'data'
}

should_hash() {
  class=$1
  case "$HASH_MODE" in
    all) return 0 ;;
    none) return 1 ;;
  esac
  case "$class" in
    mach-o|native-library|shell|python|javascript|typescript|json|plist|package|disk-image|archive|key-material|executable-other) return 0 ;;
  esac
  return 1
```

```
}

static_parse() {
  class=$1
  path=$2
  detail=$3
  case "$class" in
    shell)
      if /bin/bash -n "$path" > "$detail" 2>&1; then printf 'pass'; else printf 'fail'; fi
      ;;
    python)
      if python3 - "$path" > "$detail" 2>&1 <<'PY'
import ast
import sys
import tokenize

path = sys.argv[1]
with tokenize.open(path) as handle:
    ast.parse(handle.read(), filename=path)
PY
      then printf 'pass'; else printf 'fail'; fi
      ;;
    javascript)
      if command -v node >/dev/null 2>&1; then
        if node --check "$path" > "$detail" 2>&1; then printf 'pass'; else printf 'fail'; fi
```

```
      else
        printf 'not_checked_node_missing'
      fi
      ;;
    json)
      if command -v jq >/dev/null 2>&1; then
        if jq empty "$path" > "$detail" 2>&1; then printf 'pass'; else printf 'fail'; fi
      elif plutil -lint "$path" > "$detail" 2>&1; then printf 'pass'; else printf 'fail'; fi
      ;;
    plist)
      if plutil -lint "$path" > "$detail" 2>&1; then printf 'pass'; else printf 'fail'; fi
      ;;
    typescript)
      printf 'not_checked_project_compiler_required'
      ;;
    *)
      printf 'not_applicable'
      ;;
  esac
}

verify_code_file() {
  path=$1
  rel=$2
  class=$3
  sha=$4
```

```
top_dir=$5
item_id="$(path_id "$rel")_$(safe_component "$(basename "$rel")")"
detail_rel="details/${item_id}.txt"
detail="$top_dir/$detail_rel"
: > "$detail"

parse_status="$(static_parse "$class" "$path" "$detail")"
codesign_status="not_applicable"
gatekeeper_status="not_applicable"
identifier=""
team_identifier=""
authorities=""

case "$class" in
  mach-olnative-library)
    {
      echo '[codesign verify]'
      codesign --verify --strict --verbose=4 "$path"
    } >> "$detail" 2>&1
    rc=$?
    if [ "$rc" -eq 0 ]; then
      codesign_status="valid"
    elif rg -q 'not signed at alllcode object is not signed' "$detail" 2>/dev/null; then
      codesign_status="unsigned"
    else
```

```
      codesign_status="invalid"
    fi
    {
      echo
      echo '[codesign metadata]'
      codesign -dvvv "$path"
      echo
      echo '[entitlements]'
      codesign -d --entitlements :- "$path"
    } >> "$detail" 2>&1 || true
    identifier=$(awk -F= '/^Identifier=/{print $2; exit}' "$detail")
    team_identifier=$(awk -F= '/^TeamIdentifier=/{print $2; exit}' "$detail")
    authorities=$(awk -F= '/^Authority=/{if (n++) printf ";"; printf "%s",$2}' "$detail")
    ;;
esac

xattr_names=$(xattr "$path" 2>/dev/null | paste -sd, -)
quarantine=$(xattr -p com.apple.quarantine "$path" 2>/dev/null | tr '\t\r\n' '    ' || true)
[ -s "$detail" ] || rm -f "$detail"
[ -e "$detail" ] || detail_rel=""

printf '%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\n' \
  "$(escape_tsv "$rel")" "$(escape_tsv "$class")" "$(escape_tsv "$sha")" \
  "$(escape_tsv "$parse_status")" "$(escape_tsv "$codesign_status")" \
  "$(escape_tsv "$gatekeeper_status")" "$(escape_tsv "$identifier")" \
  "$(escape_tsv "$team_identifier")" "$(escape_tsv "$authorities")" \
```

```
    "$(escape_tsv "$xattr_names")" "$(escape_tsv "$quarantine")" \
    "$(escape_tsv "$detail_rel")" >> "$top_dir/code_verification.tsv"
}

verify_app_bundles() {
  volume=$1
  volume_out=$2
  app_report="$volume_out/app_bundles.tsv"
  printf 'relative_path\tcodesign\tgatekeeper\tidentifier\tteam_identifier\tauthorities\tdetail_file\n' >
"$app_report"
  while IFS= read -r -d '' app; do
    rel=$(relative_path "$app" "$volume")
    item_id="$(path_id "$rel")_$(safe_component "$(basename "$rel")")"
    detail_rel="bundle_details/${item_id}.txt"
    detail="$volume_out/$detail_rel"
    mkdir -p "$(dirname "$detail")"
    {
      echo '[bundle codesign verify]'
      codesign --verify --deep --strict --verbose=4 "$app"
    } > "$detail" 2>&1
    rc=$?
    if [ "$rc" -eq 0 ]; then sign_status="valid"; else sign_status="invalid"; fi
    {
      echo
      echo '[bundle codesign metadata]'
      codesign -dvvv "$app"
```

```
      echo
      echo '[bundle entitlements]'
      codesign -d --entitlements :- "$app"
      echo
      echo '[gatekeeper]'
      spctl --assess --type execute -vv "$app"
    } >> "$detail" 2>&1
    spctl --assess --type execute -vv "$app" >/dev/null 2>&1
    if [ "$?" -eq 0 ]; then gatekeeper="accepted"; else gatekeeper="rejected_or_unavailable"; fi
    identifier=$(awk -F= '/^Identifier=/{print $2; exit}' "$detail")
    team_identifier=$(awk -F= '/^TeamIdentifier=/{print $2; exit}' "$detail")
    authorities=$(awk -F= '/^Authority=/{if (n++) printf ";"; printf "%s",$2}' "$detail")
    printf '%s\t%s\t%s\t%s\t%s\t%s\t%s\n' \
      "$(escape_tsv "$rel")" "$sign_status" "$gatekeeper" \
      "$(escape_tsv "$identifier")" "$(escape_tsv "$team_identifier")" \
      "$(escape_tsv "$authorities")" "$(escape_tsv "$detail_rel")" >> "$app_report"
  done < <(find "$volume" -xdev -type d -name '*.app' -prune -print0 2>> "$volume_out/find_errors.log")
}

inspect_dmg_contents() {
  path=$1
  outer_rel=$2
  volume_out=$3
  item_id=$4
```

```
content_out="$volume_out/container_details/${item_id}_contents"
mkdir -p "$content_out"
attach_log="$content_out/attach.txt"
if ! hdiutil attach -readonly -nobrowse -noautoopen -owners off "$path" > "$attach_log" 2>&1; then
  printf 'attach_failed'
  return
fi

backing_device=$(awk '/^\/dev\/disk[0-9]+[[:space:]]/{print $1; exit}' "$attach_log")
mount_point=$(awk -F '\t' 'index($NF,"/Volumes/")==1{print $NF; exit}' "$attach_log")
if [ -n "$backing_device" ]; then
  ATTACHED_DEVICES+=("$backing_device")
fi
if [ -z "$backing_device" ] || [ -z "$mount_point" ] || [ ! -d "$mount_point" ]; then
  if [ -n "$backing_device" ]; then
    hdiutil detach "$backing_device" >> "$attach_log" 2>&1 || true
    mark_detached "$backing_device"
  fi
  printf 'attach_missing_mount'
  return
fi

mount_line=$(mount | grep -F " on $mount_point " | head -n 1 || true)
printf '\n[mount]\n%s\n' "$mount_line" >> "$attach_log"
case "$mount_line" in
  *read-only*) ;;
```

```
  *)
    hdiutil detach "$backing_device" >> "$attach_log" 2>&1 || true
    mark_detached "$backing_device"
    printf 'rejected_not_read_only'
    return
    ;;
esac

objects="$content_out/objects.tsv"
printf 'internal_path\tkind\tmode\tsize\tsha256\tclass\tstatic_parse\tfile_description\n' > "$objects"
object_count=0
partial=0
while IFS= read -r -d '' internal_path; do
  object_count=$((object_count + 1))
  if [ "$DMG_MAX_FILES" -gt 0 ] && [ "$object_count" -gt "$DMG_MAX_FILES" ]; then
    partial=1
    break
  fi
  internal_rel=$(relative_path "$internal_path" "$mount_point")
  kind="other"
  mode=$(stat -f '%Sp' "$internal_path" 2>/dev/null || true)
  size=$(stat -f '%z' "$internal_path" 2>/dev/null || true)
  sha=""
  class="not_applicable"
  parse_status="not_applicable"
```

```
  description=""
  if [ -L "$internal_path" ]; then
    kind="symlink"
    description=$(file -b -h "$internal_path" 2>/dev/null || true)
  elif [ -d "$internal_path" ]; then
    kind="directory"
    description="directory"
  elif [ -f "$internal_path" ]; then
    kind="file"
    description=$(file -b "$internal_path" 2>/dev/null || true)
    class=$(classify_file "$internal_path" "$description")
    if should_hash "$class"; then
      sha=$(shasum -a 256 "$internal_path" 2>/dev/null | awk '{print $1}')
    fi
    case "$class" in
      shell|python|javascript|typescript|json|plist)
        parse_detail="$content_out/parse_$(path_id "$internal_rel").txt"
        parse_status=$(static_parse "$class" "$internal_path" "$parse_detail")
        [ -s "$parse_detail" ] || rm -f "$parse_detail"
        ;;
    esac
  fi
  printf '%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\n' \
    "$(escape_tsv "$internal_rel")" "$kind" "$(escape_tsv "$mode")" \
    "$(escape_tsv "$size")" "$sha" "$class" "$parse_status" \
    "$(escape_tsv "$description")" >> "$objects"
```

```
  done < <(find "$mount_point" -xdev -print0 2>> "$content_out/find_errors.log")

  printf 'outer_path=%s\nmount_point=%s\nobjects_seen=%s\npartial=%s\n' \
    "$outer_rel" "$mount_point" "$object_count" "$partial" > "$content_out/scope.txt"
  verify_app_bundles "$mount_point" "$content_out"
  verify_containers "$mount_point" "$content_out" 0
  find "$content_out" -type f ! -name output_hashes.sha256 -exec shasum -a 256 {} \; > "$content_out/
output_hashes.sha256"

  if hdiutil detach "$backing_device" >> "$attach_log" 2>&1; then
    mark_detached "$backing_device"
    if [ "$partial" -eq 1 ]; then printf 'partial'; else printf 'inventoried'; fi
  else
    printf 'inventoried_detach_failed'
  fi
}

verify_containers() {
  volume=$1
  volume_out=$2
  allow_content_inspection=${3:-1}
  report="$volume_out/container_verification.tsv"
  printf
'relative_path\tclass\tsha256\tstructure\tchecksum\tsignature\tgatekeeper\ttrust\tcontent_inspection\tdetail
_file\n' > "$report"
  while IFS= read -r -d '' path; do
    rel=$(relative_path "$path" "$volume")
```

```
lower=$(printf '%s' "$path" | tr '[:upper:]' '[:lower:]')
sha=$(shasum -a 256 "$path" 2>/dev/null | awk '{print $1}')
item_id="$(path_id "$rel")_$(safe_component "$(basename "$rel")")"
detail_rel="container_details/${item_id}.txt"
detail="$volume_out/$detail_rel"
mkdir -p "$(dirname "$detail")"
structure="not_checked"
checksum="not_applicable"
signature="not_applicable"
gatekeeper="not_applicable"
trust="not_assessed"
content_inspection="not_requested"
class="archive"
case "$lower" in
  *.pkg)
    class="package"
    {
      echo '[pkg signature]'
      pkgutil --check-signature "$path"
      echo
      echo '[gatekeeper]'
      spctl --assess --type install -vv "$path"
      echo
      echo '[xar listing: first 5000 entries]'
      xar -tf "$path" | awk 'NR<=5000'
```

```
    } > "$detail" 2>&1

    if pkgutil --check-signature "$path" >/dev/null 2>&1; then signature="valid"; else
signature="invalid_or_unsigned"; fi

    if spctl --assess --type install -vv "$path" >/dev/null 2>&1; then gatekeeper="accepted"; else
gatekeeper="rejected"; fi

    if xar -tf "$path" >/dev/null 2>&1; then structure="listable"; else structure="invalid"; fi

    if [ "$signature" = "valid" ] && [ "$gatekeeper" = "accepted" ]; then trust="trusted"; else
trust="untrusted"; fi

    ;;

  *.dmg)

    class="disk-image"

    {

      echo '[image info]'

      hdiutil imageinfo "$path"

      echo

      echo '[image verify]'

      hdiutil verify "$path"

      echo

      echo '[codesign verify]'

      codesign --verify --strict --verbose=4 "$path"

      echo

      echo '[codesign metadata]'

      codesign -dvvv "$path"

      echo

      echo '[gatekeeper primary signature]'

      spctl --assess --type open --context context:primary-signature -vv "$path"

    } > "$detail" 2>&1
```

```
    if hdiutil imageinfo "$path" >/dev/null 2>&1; then structure="valid"; else structure="invalid"; fi

    if hdiutil verify "$path" >/dev/null 2>&1; then checksum="valid"; else checksum="invalid"; fi

    if codesign --verify --strict --verbose=4 "$path" >/dev/null 2>&1; then

      signature="valid"

    else

      codesign_description=$(codesign -dvvv "$path" 2>&1 || true)

      if printf '%s\n' "$codesign_description" | rg -q 'not signed at all|code object is not signed'; then

        signature="unsigned"

      else

        signature="invalid"

      fi

    fi

    if spctl --assess --type open --context context:primary-signature -vv "$path" >/dev/null 2>&1; then
gatekeeper="accepted"; else gatekeeper="rejected"; fi

    if [ "$structure" != "valid" ]; then

      trust="untrusted_invalid_structure"

    elif [ "$checksum" != "valid" ]; then

      trust="untrusted_checksum_failed"

    elif [ "$signature" = "unsigned" ]; then

      trust="untrusted_unsigned"

    elif [ "$signature" != "valid" ]; then

      trust="untrusted_invalid_signature"

    elif [ "$gatekeeper" != "accepted" ]; then

      trust="untrusted_gatekeeper_rejected"

    else

      trust="trusted"
```

```
        fi

        if [ "$INSPECT_DMG_CONTENTS" -eq 1 ] && [ "$allow_content_inspection" -eq 1 ] && [ "$structure"
= "valid" ]; then

          content_inspection=$(inspect_dmg_contents "$path" "$rel" "$volume_out" "$item_id")

        fi

        ;;

      *.zip)

        unzip -Z1 "$path" 2>&1 | awk 'NR<=5000' > "$detail"

        [ "${PIPESTATUS[0]}" -eq 0 ] && structure="listable" || structure="invalid"

        trust="untrusted_unsigned_archive"

        ;;

      *)

        tar -tf "$path" 2>&1 | awk 'NR<=5000' > "$detail"

        [ "${PIPESTATUS[0]}" -eq 0 ] && structure="listable" || structure="invalid"

        trust="untrusted_unsigned_archive"

        ;;

    esac

    printf '%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\n' \

      "$(escape_tsv "$rel")" "$class" "$sha" "$structure" "$checksum" \

      "$signature" "$gatekeeper" "$trust" "$content_inspection" \

      "$(escape_tsv "$detail_rel")" >> "$report"

  done < <(

    if [ -n "$CONTAINER_ONLY" ] && [ "$allow_content_inspection" -eq 1 ]; then

      printf '%s\0' "$CONTAINER_ONLY"

    else

      find "$volume" -xdev -type f \( -iname '*.pkg' -o -iname '*.dmg' -o -iname '*.zip' -o -iname '*.tar' -o
```

```
-print0 2>> "$volume_out/find_errors.log"
    fi
  )
}

verify_container_only() {
  path=$1
  source_dir=$(dirname "$path")
  source_mount=$(df "$path" 2>/dev/null | awk 'NR==2{print $NF}')
  source_mount_line=$(mount | grep -F " on $source_mount " | head -n 1 || true)
  if [ "$ALLOW_WRITABLE" -ne 1 ]; then
    case "$source_mount_line" in
      *read-only*) ;;
      *) fatal "refusing container on writable or unverified source: $path" ;;
    esac
  fi

  slug="container_only_$(path_id "$path")"
  volume_out="$OUT/$slug"
  mkdir -p "$volume_out"
  printf '%s\n' "$source_mount_line" > "$volume_out/00_mount.txt"
  ls -laOe@ "$path" > "$volume_out/00_source_listing.txt" 2>&1 || true
  verify_containers "$source_dir" "$volume_out" 1
  find "$volume_out" -type f ! -name output_hashes.sha256 -exec shasum -a 256 {} \; > "$volume_out/
output_hashes.sha256"
}
```

```
collect_high_signal_paths() {
  volume=$1
  volume_out=$2
  find "$volume" -xdev \( \
    -path '*/LaunchAgents/*' -o -path '*/LaunchDaemons/*' -o \
    -path '*/ConfigurationProfiles/*' -o -path '*/Managed Preferences/*' -o \
    -name 'mdmclient.plist' -o -name 'CloudConfigurationDetails.plist' -o \
    -name 'PayloadManifest.plist' -o -name 'TCC.db' -o -name 'TCCAccessory.db' -o \
    -name 'appsscript.json' -o -name '.clasprc.json' -o -name 'workspace_admin_audit.json' \
  \) -print 2>> "$volume_out/find_errors.log" > "$volume_out/high_signal_paths.txt"

  if command -v rg >/dev/null 2>&1; then
    rg --hidden --no-follow --no-messages --with-filename -i -n -o \
      --max-filesize "${MAX_TEXT_MB}M" \
      -g '!**/.git/objects/**' -g '!**/node_modules/**' -g '!**/.venv/**' \
      -g '!**/venv/**' -g '!**/Library/Caches/**' -g '!**/homebrew/Cellar/**' \
      -g '!*.jmod' -g '!*.jar' -g '!*.zip' -g '!*.tar*' -g '!*.dmg' -g '!*.pkg' \
      -e 'token|secret|api[_-]?key|password|passphrase|private[ _-]?key|BEGIN (RSA |OPENSSH |EC )?
PRIVATE KEY' \
      "$volume" > "$volume_out/sensitive_keyword_hits_redacted.txt" 2>> "$volume_out/rg_errors.log" ||
true

    rg --hidden --no-follow --no-messages --with-filename -i -n -o \
      --max-filesize "${MAX_TEXT_MB}M" \
      -g '!**/.git/objects/**' -g '!**/node_modules/**' -g '!**/.venv/**' \
      -g '!**/venv/**' -g '!**/Library/Caches/**' -g '!**/homebrew/Cellar/**' \
```

```
    -g '!*.jmod' -g '!*.jar' -g '!*.zip' -g '!*.tar*' -g '!*.dmg' -g '!*.pkg' \

    -e 'mdmBaseURL|axm-servicediscovery|com\.apple\.remotemanagement|ConfigurationProfiles|
RunAtLoad|KeepAlive|ProgramArguments|osascript|curl[[:space:]]|base64|cloudflare|warp|codex-local|
unsloth|gemma' \

    "$volume" > "$volume_out/forensic_keyword_hits.txt" 2>> "$volume_out/rg_errors.log" || true

  else

    printf 'ripgrep unavailable; text search not run\n' > "$volume_out/rg_errors.log"

  fi

}



summarize_directories() {

  volume_out=$1

  report="$volume_out/directory_summary.tsv"

  printf 'directory_group\tobjects\tcode_candidates\tparse_failures\tinvalid_or_unsigned_native_code\n' >
"$report"

  for dir in "$volume_out"/directories/*; do

    [ -d "$dir" ] || continue

    objects=$(awk 'NR>1{n++} END{print n+0}' "$dir/objects.tsv")

    candidates=$(awk 'NR>1{n++} END{print n+0}' "$dir/code_verification.tsv")

    parse_failures=$(awk -F '\t' 'NR>1 && $4=="fail"{n++} END{print n+0}' "$dir/code_verification.tsv")

    native_failures=$(awk -F '\t' 'NR>1 && ($5=="invalid" || $5=="unsigned"){n++} END{print n+0}' "$dir/
code_verification.tsv")

    printf '%s\t%s\t%s\t%s\t%s\n' "$(basename "$dir")" "$objects" "$candidates" "$parse_failures"
"$native_failures" >> "$report"

  done

}



inventory_volume() {
```

```
  volume=$1
  [ -d "$volume" ] || fatal "source volume is not a directory: $volume"
  volume="$(cd "$volume" && pwd -P)"
  case "$OUT/" in "$volume/"*) fatal "output directory cannot be inside source volume: $volume";; esac

  vname="$(basename "$volume")"
  vslug="$(safe_component "$vname")_$(path_id "$volume")"
  CURRENT_VOUT="$OUT/$vslug"
  export CURRENT_VOUT
  mkdir -p "$CURRENT_VOUT/directories"

  source_mount=$(df "$volume" 2>/dev/null | awk 'NR==2{print $NF}')
  mount_line=$(mount | grep -F " on $source_mount " | head -n 1 || true)
  disk_read_only=$(diskutil info "$source_mount" 2>/dev/null | awk -F: '/Volume Read-Only/{gsub(/^[ \t]
+/,"",$2); print $2; exit}')
  if [ "$ALLOW_WRITABLE" -ne 1 ]; then
    case "$mount_line $disk_read_only" in
      *read-only*|*'Yes (read-only mount flag set)'*) ;;
      *) fatal "refusing writable or unverified source: $volume" ;;
    esac
  fi

  log "Starting volume: $volume"
  date -u +%Y-%m-%dT%H:%M:%SZ > "$CURRENT_VOUT/00_utc_start.txt"
  printf '%s\n' "$mount_line" > "$CURRENT_VOUT/00_mount.txt"
  diskutil info "$source_mount" > "$CURRENT_VOUT/00_diskutil_info.txt" 2>&1 || true
```

```
ls -laOe@ "$volume" > "$CURRENT_VOUT/00_root_listing.txt" 2>&1 || true

file_count=0
partial=0
while IFS= read -r -d '' path; do
  rel=$(relative_path "$path" "$volume")
  top=$(top_name "$rel")
  top_dir=$(top_output_dir "$top")
  kind="other"
  description=""
  link_target=""
  sha=""
  class="not_applicable"

  if [ -L "$path" ]; then
    kind="symlink"
    link_target=$(readlink "$path" 2>/dev/null || true)
    description=$(file -b -h "$path" 2>/dev/null || true)
  elif [ -d "$path" ]; then
    kind="directory"
    description="directory"
  elif [ -f "$path" ]; then
    kind="file"
    file_count=$((file_count + 1))
    if [ "$LIMIT_FILES" -gt 0 ] && [ "$file_count" -gt "$LIMIT_FILES" ]; then
```

```
      partial=1
      break
    fi
    description=$(file -b "$path" 2>/dev/null || true)
    class=$(classify_file "$path" "$description")
    if should_hash "$class"; then
      sha=$(shasum -a 256 "$path" 2>/dev/null | awk '{print $1}')
      [ -n "$sha" ] || record_error "sha256" "$rel" "1" "hash failed"
    fi
  fi

  stat_fields=$(stat -f '%Sp|%u|%g|%z|%m|%B|%Sf' "$path" 2>/dev/null || true)
  mode=$(printf '%s' "$stat_fields" | awk -F '|' '{print $1}')
  uid=$(printf '%s' "$stat_fields" | awk -F '|' '{print $2}')
  gid=$(printf '%s' "$stat_fields" | awk -F '|' '{print $3}')
  size=$(printf '%s' "$stat_fields" | awk -F '|' '{print $4}')
  mtime=$(printf '%s' "$stat_fields" | awk -F '|' '{print $5}')
  birth=$(printf '%s' "$stat_fields" | awk -F '|' '{print $6}')
  flags=$(printf '%s' "$stat_fields" | awk -F '|' '{print $7}')
  write_object_row "$top_dir/objects.tsv" "$rel" "$kind" "$mode" "$uid" "$gid" "$size" "$mtime" "$birth"
"$flags" "$sha" "$class" "$description" "$link_target"

  case "$class" in
    mach-o|native-library|shell|python|javascript|typescript|json|plist|executable-other)
      verify_code_file "$path" "$rel" "$class" "$sha" "$top_dir"
      ;;
```

```
    esac

  done < <(find "$volume" -xdev -print0 2>> "$CURRENT_VOUT/find_errors.log")


  printf 'file_limit=%s\nfiles_seen=%s\npartial=%s\n' "$LIMIT_FILES" "$file_count" "$partial" >
"$CURRENT_VOUT/run_scope.txt"

  verify_app_bundles "$volume" "$CURRENT_VOUT"

  verify_containers "$volume" "$CURRENT_VOUT"

  collect_high_signal_paths "$volume" "$CURRENT_VOUT"

  summarize_directories "$CURRENT_VOUT"


  date -u +%Y-%m-%dT%H:%M:%SZ > "$CURRENT_VOUT/00_utc_end.txt"

  log "Finished volume: $volume (files seen: $file_count, partial: $partial)"

  find "$CURRENT_VOUT" -type f ! -name output_hashes.sha256 -exec shasum -a 256 {} \; >
"$CURRENT_VOUT/output_hashes.sha256"

}


{

  echo "case=$CASE"

  echo "started_utc=$(date -u +%Y-%m-%dT%H:%M:%SZ)"

  echo "script=$0"

  echo "script_sha256=$(shasum -a 256 "$0" | awk '{print $1}')"

  echo "hash_mode=$HASH_MODE"

  echo "max_text_mb=$MAX_TEXT_MB"

  echo "limit_files=$LIMIT_FILES"

  echo "inspect_dmg_contents=$INSPECT_DMG_CONTENTS"

  echo "dmg_max_files=$DMG_MAX_FILES"

  echo "container_only=$CONTAINER_ONLY"
```

```
  echo "allow_writable=$ALLOW_WRITABLE"
  for source_volume in "${VOLUMES[@]-}"; do
    [ -n "$source_volume" ] && printf 'source=%s\n' "$source_volume"
  done
} > "$OUT/00_case_manifest.txt"

{
  bash --version | head -n 1
  shasum -a 256 "$0"
  file --version 2>&1 | head -n 1
  codesign --version 2>&1 || true
  spctl --version 2>&1 || true
  plutil -help 2>&1 | head -n 1
  rg --version 2>&1 | head -n 1 || true
  jq --version 2>&1 || true
  python3 --version 2>&1 || true
  node --version 2>&1 || true
} > "$OUT/00_tool_versions.txt"

log "Case output: $OUT"
if [ -n "$CONTAINER_ONLY" ]; then
  verify_container_only "$CONTAINER_ONLY"
else
  for volume in "${VOLUMES[@]}"; do
    inventory_volume "$volume"
```

```
  done

fi


RUN_COMPLETE=1

rm -f "$OUT/INCOMPLETE"

touch "$OUT/COMPLETE"

date -u +%Y-%m-%dT%H:%M:%SZ > "$OUT/00_utc_end.txt"

log "Collection complete: $OUT"

find "$OUT" -type f ! -name case_output_hashes.sha256 -exec shasum -a 256 {} \; > "$OUT/
case_output_hashes.sha256"#!/bin/bash

# Read-only, per-object verification for mounted macOS evidence volumes.

#

# Default:

#   /Volumes/Ellis/scripts/os_boot_verify_v2.sh /Volumes/OS_BOOT /Volumes/RESCUE_OS

#

# This script never executes code from the evidence volume. It inventories every

# object, hashes and statically checks code-like files individually, validates

# app/package/disk-image containers, and writes all results outside the source.


set -u

set -o pipefail


PATH="/usr/bin:/bin:/usr/sbin:/sbin:$HOME/.homebrew/bin:$PATH"

export PATH LC_ALL=C LANG=C


SCRIPT_HOME="$(cd -- "$(dirname -- "$0")" && pwd -P)"
```

```
OUT_BASE="$SCRIPT_HOME/results"
ALLOW_WRITABLE=0
HASH_MODE="code"
MAX_TEXT_MB=16
LIMIT_FILES=0
INSPECT_DMG_CONTENTS=0
DMG_MAX_FILES=100000
CONTAINER_ONLY=""
VOLUMES=()
ATTACHED_DEVICES=()

usage() {
  cat <<'EOF'
Usage: os_boot_verify_v2.sh [options] [VOLUME ...]

Options:
  --out-base DIR       Result parent directory (default: script/results)
  --hash-all           SHA-256 every regular file, including non-code data
  --no-hash            Do not hash regular files
  --max-text-mb N      Maximum text-file size searched by ripgrep (default: 16)
  --limit-files N      Stop after N regular files per volume; marks run partial
  --inspect-dmg-contents
                       Mount DMGs read-only, inventory internal objects, and
                       verify nested apps/containers without executing content
  --dmg-max-files N    Maximum internal objects inventoried per DMG (default: 100000)
  --container-only FILE
```

```
                    Verify one package/archive/DMG without walking its volume
  --allow-writable     Permit a writable source (intended only for test fixtures)
  -h, --help           Show this help

With no VOLUME arguments, mounted /Volumes/OS_BOOT and /Volumes/RESCUE_OS are used.
EOF
}

fatal() {
  echo "[FATAL] $*" >&2
  exit 2
}

while [ "$#" -gt 0 ]; do
  case "$1" in
    --out-base)
      [ "$#" -ge 2 ] ll fatal "--out-base requires a directory"
      OUT_BASE="$2"
      shift 2
      ;;
    --hash-all)
      HASH_MODE="all"
      shift
      ;;
    --no-hash)
```

```
  HASH_MODE="none"
  shift
  ;;
--max-text-mb)
  [ "$#" -ge 2 ] || fatal "--max-text-mb requires a number"
  MAX_TEXT_MB="$2"
  shift 2
  ;;
--limit-files)
  [ "$#" -ge 2 ] || fatal "--limit-files requires a number"
  LIMIT_FILES="$2"
  shift 2
  ;;
--inspect-dmg-contents)
  INSPECT_DMG_CONTENTS=1
  shift
  ;;
--dmg-max-files)
  [ "$#" -ge 2 ] || fatal "--dmg-max-files requires a number"
  DMG_MAX_FILES="$2"
  shift 2
  ;;
--container-only)
  [ "$#" -ge 2 ] || fatal "--container-only requires a file"
  CONTAINER_ONLY="$2"
  shift 2
```

```
      ;;
    --allow-writable)
      ALLOW_WRITABLE=1
      shift
      ;;
    -hl--help)
      usage
      exit 0
      ;;
    --)
      shift
      while [ "$#" -gt 0 ]; do
        VOLUMES[${#VOLUMES[@]}]="$1"
        shift
      done
      ;;
    -*)
      fatal "unknown option: $1"
      ;;
    *)
      VOLUMES[${#VOLUMES[@]}]="$1"
      shift
      ;;
  esac
done
```

```
case "$MAX_TEXT_MB" in *[!0-9]*|'') fatal "--max-text-mb must be an integer";; esac
case "$LIMIT_FILES" in *[!0-9]*|'') fatal "--limit-files must be an integer";; esac
case "$DMG_MAX_FILES" in *[!0-9]*|'') fatal "--dmg-max-files must be an integer";; esac

if [ "${#VOLUMES[@]}" -eq 0 ] && [ -z "$CONTAINER_ONLY" ]; then
  [ -d /Volumes/OS_BOOT ] && VOLUMES[${#VOLUMES[@]}]="/Volumes/OS_BOOT"
  [ -d /Volumes/RESCUE_OS ] && VOLUMES[${#VOLUMES[@]}]="/Volumes/RESCUE_OS"
fi
if [ -z "$CONTAINER_ONLY" ]; then
  [ "${#VOLUMES[@]}" -gt 0 ] || fatal "no source volumes were supplied or mounted"
else
  [ -f "$CONTAINER_ONLY" ] || fatal "container is not a regular file: $CONTAINER_ONLY"
  CONTAINER_ONLY="$(cd -- "$(dirname -- "$CONTAINER_ONLY")" && pwd -P)/$(basename --
"$CONTAINER_ONLY")"
fi

mkdir -p "$OUT_BASE" || fatal "cannot create output base: $OUT_BASE"
OUT_BASE="$(cd "$OUT_BASE" && pwd -P)"
UTC_STAMP="$(date -u +%Y%m%dT%H%M%SZ)"
CASE="USB_verify_v2_${UTC_STAMP}"
OUT="$OUT_BASE/$CASE"
mkdir -p "$OUT"
touch "$OUT/INCOMPLETE"

RUN_COMPLETE=0
```

```
on_exit() {
  rc=$?
  for attached_device in "${ATTACHED_DEVICES[@]-}"; do
    [ -n "$attached_device" ] || continue
    hdiutil detach "$attached_device" >/dev/null 2>&1 || true
  done
  if [ "$RUN_COMPLETE" -eq 1 ]; then
    rm -f "$OUT/INCOMPLETE"
    touch "$OUT/COMPLETE"
  else
    date -u +%Y-%m-%dT%H:%M:%SZ > "$OUT/00_utc_end.txt" 2>/dev/null || true
    printf 'exit_code=%s\n' "$rc" > "$OUT/INTERRUPTED.txt" 2>/dev/null || true
  fi
  trap - EXIT
  exit "$rc"
}

mark_detached() {
  detached_device=$1
  for attached_index in "${!ATTACHED_DEVICES[@]}"; do
    if [ "${ATTACHED_DEVICES[$attached_index]}" = "$detached_device" ]; then
      unset 'ATTACHED_DEVICES[attached_index]'
    fi
  done
}
trap on_exit EXIT
```

```
trap 'exit 130' INT
trap 'exit 143' TERM

LOG="$OUT/command_log.md"
ERRORS="$OUT/errors.tsv"
printf 'stage\tpath\texit_code\tmessage\n' > "$ERRORS"

log() {
  now="$(date -u +%Y-%m-%dT%H:%M:%SZ)"
  printf '[%s] %s\n' "$now" "$*" | tee -a "$LOG"
}

escape_tsv() {
  value=$1
  value=${value//\\/\\\\}
  value=${value//$'\t'/\\t}
  value=${value//$'\r'/\\r}
  value=${value//$'\n'/\\n}
  printf '%s' "$value"
}

record_error() {
  stage=$(escape_tsv "$1")
  path=$(escape_tsv "$2")
  code=$(escape_tsv "$3")
```

```
  message=$(escape_tsv "$4")
  printf '%s\t%s\t%s\t%s\n' "$stage" "$path" "$code" "$message" >> "$ERRORS"
}

safe_component() {
  printf '%s' "$1" | tr -cs 'A-Za-z0-9._-' '_' | cut -c1-80
}

path_id() {
  printf '%s' "$1" | shasum -a 256 | awk '{print substr($1,1,16)}'
}

relative_path() {
  path=$1
  root=$2
  if [ "$path" = "$root" ]; then
    printf '.'
  else
    printf '%s' "${path#"$root"/}"
  fi
}

top_name() {
  rel=$1
  case "$rel" in
    */*) printf '%s' "${rel%%/*}" ;;
```

```
    .) printf '_ROOT_' ;;
    *) printf '_ROOT_FILES' ;;
  esac
}

top_output_dir() {
  top=$1
  slug="$(safe_component "$top")_$(path_id "$top")"
  dir="$CURRENT_VOUT/directories/$slug"
  mkdir -p "$dir/details"
  if [ ! -e "$dir/objects.tsv" ]; then
    printf 'relative_path\tkind\tmode\tuid\tgid\tsize\tmtime_epoch\tbirth_epoch\tflags\tsha256\tclass\tfile_description\tlink_target\n' > "$dir/objects.tsv"
  fi
  if [ ! -e "$dir/code_verification.tsv" ]; then
    printf 'relative_path\tclass\tsha256\tstatic_parse\tcodesign\tgatekeeper\tidentifier\tteam_identifier\tauthorities\txattr_names\tquarantine\tdetail_file\n' > "$dir/code_verification.tsv"
  fi
  printf '%s' "$dir"
}

write_object_row() {
  out_file=$1
  rel=$2
  kind=$3
```

```
  mode=$4
  uid=$5
  gid=$6
  size=$7
  mtime=$8
  birth=$9
  flags=${10}
  sha=${11}
  class=${12}
  description=${13}
  link_target=${14}
  printf '%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\n' \
    "$(escape_tsv "$rel")" "$(escape_tsv "$kind")" "$(escape_tsv "$mode")" \
    "$(escape_tsv "$uid")" "$(escape_tsv "$gid")" "$(escape_tsv "$size")" \
    "$(escape_tsv "$mtime")" "$(escape_tsv "$birth")" "$(escape_tsv "$flags")" \
    "$(escape_tsv "$sha")" "$(escape_tsv "$class")" "$(escape_tsv "$description")" \
    "$(escape_tsv "$link_target")" >> "$out_file"
}

classify_file() {
  path=$1
  description=$2
  lower=$(printf '%s' "$path" | tr '[:upper:]' '[:lower:]')
  case "$description" in
    *Mach-O*) printf 'mach-o'; return ;;
  esac
```

```
case "$lower" in
  *.dylibl*.sol*.bundle) printf 'native-library'; return ;;
  *.shl*.bashl*.zshl*.command) printf 'shell'; return ;;
  *.pyl*.pyw) printf 'python'; return ;;
  *.jsl*.mjsl*.cjs) printf 'javascript'; return ;;
  *.tsl*.tsx) printf 'typescript'; return ;;
  *.json) printf 'json'; return ;;
  *.plist) printf 'plist'; return ;;
  *.pkg) printf 'package'; return ;;
  *.dmg) printf 'disk-image'; return ;;
  *.zipl*.tarl*.tgzl*.tar.gzl*.tbzl*.tbz2l*.tar.bz2l*.tar.xz) printf 'archive'; return ;;
  *.peml*.keyl*.p12l*.pfx) printf 'key-material'; return ;;
esac
if [ -x "$path" ]; then
  if [ "$(head -c 2 "$path" 2>/dev/null)" = '#!' ]; then
    first_line=$(head -n 1 "$path" 2>/dev/null ll true)
    case "$first_line" in
      *python*) printf 'python'; return ;;
      *node*) printf 'javascript'; return ;;
      *bash*l*zsh*l*'/sh'*) printf 'shell'; return ;;
    esac
  fi
  printf 'executable-other'
  return
fi
```

```
  printf 'data'
}

should_hash() {
  class=$1
  case "$HASH_MODE" in
    all) return 0 ;;
    none) return 1 ;;
  esac
  case "$class" in
    mach-o|native-library|shell|python|javascript|typescript|json|plist|package|disk-image|archive|key-material|executable-other) return 0 ;;
  esac
  return 1
}

static_parse() {
  class=$1
  path=$2
  detail=$3
  case "$class" in
    shell)
      if /bin/bash -n "$path" > "$detail" 2>&1; then printf 'pass'; else printf 'fail'; fi
      ;;
    python)
      if python3 - "$path" > "$detail" 2>&1 <<'PY'
```

```
import ast
import sys
import tokenize

path = sys.argv[1]
with tokenize.open(path) as handle:
    ast.parse(handle.read(), filename=path)
PY
      then printf 'pass'; else printf 'fail'; fi
      ;;
    javascript)
      if command -v node >/dev/null 2>&1; then
        if node --check "$path" > "$detail" 2>&1; then printf 'pass'; else printf 'fail'; fi
      else
        printf 'not_checked_node_missing'
      fi
      ;;
    json)
      if command -v jq >/dev/null 2>&1; then
        if jq empty "$path" > "$detail" 2>&1; then printf 'pass'; else printf 'fail'; fi
      elif plutil -lint "$path" > "$detail" 2>&1; then printf 'pass'; else printf 'fail'; fi
      ;;
    plist)
      if plutil -lint "$path" > "$detail" 2>&1; then printf 'pass'; else printf 'fail'; fi
      ;;
    typescript)
```

```
      printf 'not_checked_project_compiler_required'
      ;;
    *)
      printf 'not_applicable'
      ;;
  esac
}

verify_code_file() {
  path=$1
  rel=$2
  class=$3
  sha=$4
  top_dir=$5
  item_id="$(path_id "$rel")_$(safe_component "$(basename "$rel")")"
  detail_rel="details/${item_id}.txt"
  detail="$top_dir/$detail_rel"
  : > "$detail"

  parse_status="$(static_parse "$class" "$path" "$detail")"
  codesign_status="not_applicable"
  gatekeeper_status="not_applicable"
  identifier=""
  team_identifier=""
  authorities=""
```

```
case "$class" in
  mach-olnative-library)
    {
      echo '[codesign verify]'
      codesign --verify --strict --verbose=4 "$path"
    } >> "$detail" 2>&1
    rc=$?
    if [ "$rc" -eq 0 ]; then
      codesign_status="valid"
    elif rg -q 'not signed at alllcode object is not signed' "$detail" 2>/dev/null; then
      codesign_status="unsigned"
    else
      codesign_status="invalid"
    fi
    {
      echo
      echo '[codesign metadata]'
      codesign -dvvv "$path"
      echo
      echo '[entitlements]'
      codesign -d --entitlements :- "$path"
    } >> "$detail" 2>&1 ll true
    identifier=$(awk -F= '/^Identifier=/{print $2; exit}' "$detail")
    team_identifier=$(awk -F= '/^TeamIdentifier=/{print $2; exit}' "$detail")
    authorities=$(awk -F= '/^Authority=/{if (n++) printf ";"; printf "%s",$2}' "$detail")
```

```
      ;;
  esac

  xattr_names=$(xattr "$path" 2>/dev/null | paste -sd, -)
  quarantine=$(xattr -p com.apple.quarantine "$path" 2>/dev/null | tr '\t\r\n' '   ' || true)
  [ -s "$detail" ] || rm -f "$detail"
  [ -e "$detail" ] || detail_rel=""

  printf '%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\n' \
    "$(escape_tsv "$rel")" "$(escape_tsv "$class")" "$(escape_tsv "$sha")" \
    "$(escape_tsv "$parse_status")" "$(escape_tsv "$codesign_status")" \
    "$(escape_tsv "$gatekeeper_status")" "$(escape_tsv "$identifier")" \
    "$(escape_tsv "$team_identifier")" "$(escape_tsv "$authorities")" \
    "$(escape_tsv "$xattr_names")" "$(escape_tsv "$quarantine")" \
    "$(escape_tsv "$detail_rel")" >> "$top_dir/code_verification.tsv"
}

verify_app_bundles() {
  volume=$1
  volume_out=$2
  app_report="$volume_out/app_bundles.tsv"
  printf 'relative_path\tcodesign\tgatekeeper\tidentifier\tteam_identifier\tauthorities\tdetail_file\n' >
"$app_report"
  while IFS= read -r -d '' app; do
    rel=$(relative_path "$app" "$volume")
    item_id="$(path_id "$rel")_$(safe_component "$(basename "$rel")")"
```

```
detail_rel="bundle_details/${item_id}.txt"
detail="$volume_out/$detail_rel"
mkdir -p "$(dirname "$detail")"
{
  echo '[bundle codesign verify]'
  codesign --verify --deep --strict --verbose=4 "$app"
} > "$detail" 2>&1
rc=$?
if [ "$rc" -eq 0 ]; then sign_status="valid"; else sign_status="invalid"; fi
{
  echo
  echo '[bundle codesign metadata]'
  codesign -dvvv "$app"
  echo
  echo '[bundle entitlements]'
  codesign -d --entitlements :- "$app"
  echo
  echo '[gatekeeper]'
  spctl --assess --type execute -vv "$app"
} >> "$detail" 2>&1
spctl --assess --type execute -vv "$app" >/dev/null 2>&1
if [ "$?" -eq 0 ]; then gatekeeper="accepted"; else gatekeeper="rejected_or_unavailable"; fi
identifier=$(awk -F= '/^Identifier=/{print $2; exit}' "$detail")
team_identifier=$(awk -F= '/^TeamIdentifier=/{print $2; exit}' "$detail")
authorities=$(awk -F= '/^Authority=/{if (n++) printf ";"; printf "%s",$2}' "$detail")
```

```
    printf '%s\t%s\t%s\t%s\t%s\t%s\t%s\n' \
      "$(escape_tsv "$rel")" "$sign_status" "$gatekeeper" \
      "$(escape_tsv "$identifier")" "$(escape_tsv "$team_identifier")" \
      "$(escape_tsv "$authorities")" "$(escape_tsv "$detail_rel")" >> "$app_report"
  done < <(find "$volume" -xdev -type d -name '*.app' -prune -print0 2>> "$volume_out/find_errors.log")
}

inspect_dmg_contents() {
  path=$1
  outer_rel=$2
  volume_out=$3
  item_id=$4

  content_out="$volume_out/container_details/${item_id}_contents"
  mkdir -p "$content_out"
  attach_log="$content_out/attach.txt"
  if ! hdiutil attach -readonly -nobrowse -noautoopen -owners off "$path" > "$attach_log" 2>&1; then
    printf 'attach_failed'
    return
  fi

  backing_device=$(awk '/^\/dev\/disk[0-9]+[[:space:]]/{print $1; exit}' "$attach_log")
  mount_point=$(awk -F '\t' 'index($NF,"/Volumes/")==1{print $NF; exit}' "$attach_log")
  if [ -n "$backing_device" ]; then
    ATTACHED_DEVICES+=("$backing_device")
  fi
```

```
if [ -z "$backing_device" ] || [ -z "$mount_point" ] || [ ! -d "$mount_point" ]; then
  if [ -n "$backing_device" ]; then
    hdiutil detach "$backing_device" >> "$attach_log" 2>&1 || true
    mark_detached "$backing_device"
  fi
  printf 'attach_missing_mount'
  return
fi

mount_line=$(mount | grep -F " on $mount_point " | head -n 1 || true)
printf '\n[mount]\n%s\n' "$mount_line" >> "$attach_log"
case "$mount_line" in
  *read-only*) ;;
  *)
    hdiutil detach "$backing_device" >> "$attach_log" 2>&1 || true
    mark_detached "$backing_device"
    printf 'rejected_not_read_only'
    return
    ;;
esac

objects="$content_out/objects.tsv"
printf 'internal_path\tkind\tmode\tsize\tsha256\tclass\tstatic_parse\tfile_description\n' > "$objects"
object_count=0
partial=0
```

```
while IFS= read -r -d '' internal_path; do
  object_count=$((object_count + 1))
  if [ "$DMG_MAX_FILES" -gt 0 ] && [ "$object_count" -gt "$DMG_MAX_FILES" ]; then
    partial=1
    break
  fi
  internal_rel=$(relative_path "$internal_path" "$mount_point")
  kind="other"
  mode=$(stat -f '%Sp' "$internal_path" 2>/dev/null || true)
  size=$(stat -f '%z' "$internal_path" 2>/dev/null || true)
  sha=""
  class="not_applicable"
  parse_status="not_applicable"
  description=""
  if [ -L "$internal_path" ]; then
    kind="symlink"
    description=$(file -b -h "$internal_path" 2>/dev/null || true)
  elif [ -d "$internal_path" ]; then
    kind="directory"
    description="directory"
  elif [ -f "$internal_path" ]; then
    kind="file"
    description=$(file -b "$internal_path" 2>/dev/null || true)
    class=$(classify_file "$internal_path" "$description")
    if should_hash "$class"; then
      sha=$(shasum -a 256 "$internal_path" 2>/dev/null | awk '{print $1}')
```

```
    fi
    case "$class" in
      shelllpythonljavascriptltypescriptljsonlplist)
        parse_detail="$content_out/parse_$(path_id "$internal_rel").txt"
        parse_status=$(static_parse "$class" "$internal_path" "$parse_detail")
        [ -s "$parse_detail" ] ll rm -f "$parse_detail"
        ;;
    esac
  fi
  printf '%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\n' \
    "$(escape_tsv "$internal_rel")" "$kind" "$(escape_tsv "$mode")" \
    "$(escape_tsv "$size")" "$sha" "$class" "$parse_status" \
    "$(escape_tsv "$description")" >> "$objects"
done < <(find "$mount_point" -xdev -print0 2>> "$content_out/find_errors.log")

printf 'outer_path=%s\nmount_point=%s\nobjects_seen=%s\npartial=%s\n' \
  "$outer_rel" "$mount_point" "$object_count" "$partial" > "$content_out/scope.txt"
verify_app_bundles "$mount_point" "$content_out"
verify_containers "$mount_point" "$content_out" 0
find "$content_out" -type f ! -name output_hashes.sha256 -exec shasum -a 256 {} \; > "$content_out/
output_hashes.sha256"

if hdiutil detach "$backing_device" >> "$attach_log" 2>&1; then
  mark_detached "$backing_device"
  if [ "$partial" -eq 1 ]; then printf 'partial'; else printf 'inventoried'; fi
else
```

```
    printf 'inventoried_detach_failed'
  fi
}

verify_containers() {
  volume=$1
  volume_out=$2
  allow_content_inspection=${3:-1}
  report="$volume_out/container_verification.tsv"
  printf
'relative_path\tclass\tsha256\tstructure\tchecksum\tsignature\tgatekeeper\ttrust\tcontent_inspection\tdetail
_file\n' > "$report"
  while IFS= read -r -d '' path; do
    rel=$(relative_path "$path" "$volume")
    lower=$(printf '%s' "$path" | tr '[:upper:]' '[:lower:]')
    sha=$(shasum -a 256 "$path" 2>/dev/null | awk '{print $1}')
    item_id="$(path_id "$rel")_$(safe_component "$(basename "$rel")")"
    detail_rel="container_details/${item_id}.txt"
    detail="$volume_out/$detail_rel"
    mkdir -p "$(dirname "$detail")"
    structure="not_checked"
    checksum="not_applicable"
    signature="not_applicable"
    gatekeeper="not_applicable"
    trust="not_assessed"
    content_inspection="not_requested"
```

```
    class="archive"
    case "$lower" in
      *.pkg)
        class="package"
        {
          echo '[pkg signature]'
          pkgutil --check-signature "$path"
          echo
          echo '[gatekeeper]'
          spctl --assess --type install -vv "$path"
          echo
          echo '[xar listing: first 5000 entries]'
          xar -tf "$path" | awk 'NR<=5000'
        } > "$detail" 2>&1
        if pkgutil --check-signature "$path" >/dev/null 2>&1; then signature="valid"; else
signature="invalid_or_unsigned"; fi
        if spctl --assess --type install -vv "$path" >/dev/null 2>&1; then gatekeeper="accepted"; else
gatekeeper="rejected"; fi
        if xar -tf "$path" >/dev/null 2>&1; then structure="listable"; else structure="invalid"; fi
        if [ "$signature" = "valid" ] && [ "$gatekeeper" = "accepted" ]; then trust="trusted"; else
trust="untrusted"; fi
        ;;
      *.dmg)
        class="disk-image"
        {
          echo '[image info]'
          hdiutil imageinfo "$path"
```

```
  echo
  echo '[image verify]'
  hdiutil verify "$path"
  echo
  echo '[codesign verify]'
  codesign --verify --strict --verbose=4 "$path"
  echo
  echo '[codesign metadata]'
  codesign -dvvv "$path"
  echo
  echo '[gatekeeper primary signature]'
  spctl --assess --type open --context context:primary-signature -vv "$path"
} > "$detail" 2>&1
if hdiutil imageinfo "$path" >/dev/null 2>&1; then structure="valid"; else structure="invalid"; fi
if hdiutil verify "$path" >/dev/null 2>&1; then checksum="valid"; else checksum="invalid"; fi
if codesign --verify --strict --verbose=4 "$path" >/dev/null 2>&1; then
  signature="valid"
else
  codesign_description=$(codesign -dvvv "$path" 2>&1 || true)
  if printf '%s\n' "$codesign_description" | rg -q 'not signed at all|code object is not signed'; then
    signature="unsigned"
  else
    signature="invalid"
  fi
fi
if spctl --assess --type open --context context:primary-signature -vv "$path" >/dev/null 2>&1; then
```

```
gatekeeper="accepted"; else gatekeeper="rejected"; fi

        if [ "$structure" != "valid" ]; then

          trust="untrusted_invalid_structure"

        elif [ "$checksum" != "valid" ]; then

          trust="untrusted_checksum_failed"

        elif [ "$signature" = "unsigned" ]; then

          trust="untrusted_unsigned"

        elif [ "$signature" != "valid" ]; then

          trust="untrusted_invalid_signature"

        elif [ "$gatekeeper" != "accepted" ]; then

          trust="untrusted_gatekeeper_rejected"

        else

          trust="trusted"

        fi

        if [ "$INSPECT_DMG_CONTENTS" -eq 1 ] && [ "$allow_content_inspection" -eq 1 ] && [ "$structure"
= "valid" ]; then

          content_inspection=$(inspect_dmg_contents "$path" "$rel" "$volume_out" "$item_id")

        fi

        ;;

      *.zip)

        unzip -Z1 "$path" 2>&1 | awk 'NR<=5000' > "$detail"

        [ "${PIPESTATUS[0]}" -eq 0 ] && structure="listable" || structure="invalid"

        trust="untrusted_unsigned_archive"

        ;;

      *)

        tar -tf "$path" 2>&1 | awk 'NR<=5000' > "$detail"
```

```
        [ "${PIPESTATUS[0]}" -eq 0 ] && structure="listable" || structure="invalid"
        trust="untrusted_unsigned_archive"
        ;;
    esac
    printf '%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\n' \
      "$(escape_tsv "$rel")" "$class" "$sha" "$structure" "$checksum" \
      "$signature" "$gatekeeper" "$trust" "$content_inspection" \
      "$(escape_tsv "$detail_rel")" >> "$report"
  done < <(
    if [ -n "$CONTAINER_ONLY" ] && [ "$allow_content_inspection" -eq 1 ]; then
      printf '%s\0' "$CONTAINER_ONLY"
    else
      find "$volume" -xdev -type f \( -iname '*.pkg' -o -iname '*.dmg' -o -iname '*.zip' -o -iname '*.tar' -o
-iname '*.tgz' -o -iname '*.tar.gz' -o -iname '*.tbz' -o -iname '*.tbz2' -o -iname '*.tar.bz2' -o -iname '*.tar.xz' \)
-print0 2>> "$volume_out/find_errors.log"
    fi
  )
}

verify_container_only() {
  path=$1
  source_dir=$(dirname "$path")
  source_mount=$(df "$path" 2>/dev/null | awk 'NR==2{print $NF}')
  source_mount_line=$(mount | grep -F " on $source_mount " | head -n 1 || true)
  if [ "$ALLOW_WRITABLE" -ne 1 ]; then
    case "$source_mount_line" in
      *read-only*) ;;
```

```
      *) fatal "refusing container on writable or unverified source: $path" ;;
    esac
  fi

  slug="container_only_$(path_id "$path")"
  volume_out="$OUT/$slug"
  mkdir -p "$volume_out"
  printf '%s\n' "$source_mount_line" > "$volume_out/00_mount.txt"
  ls -laOe@ "$path" > "$volume_out/00_source_listing.txt" 2>&1 || true
  verify_containers "$source_dir" "$volume_out" 1
  find "$volume_out" -type f ! -name output_hashes.sha256 -exec shasum -a 256 {} \; > "$volume_out/
output_hashes.sha256"
}

collect_high_signal_paths() {
  volume=$1
  volume_out=$2
  find "$volume" -xdev \( \
    -path '*/LaunchAgents/*' -o -path '*/LaunchDaemons/*' -o \
    -path '*/ConfigurationProfiles/*' -o -path '*/Managed Preferences/*' -o \
    -name 'mdmclient.plist' -o -name 'CloudConfigurationDetails.plist' -o \
    -name 'PayloadManifest.plist' -o -name 'TCC.db' -o -name 'TCCAccessory.db' -o \
    -name 'appsscript.json' -o -name '.clasprc.json' -o -name 'workspace_admin_audit.json' \
  \) -print 2>> "$volume_out/find_errors.log" > "$volume_out/high_signal_paths.txt"

  if command -v rg >/dev/null 2>&1; then
```

```
    rg --hidden --no-follow --no-messages --with-filename -i -n -o \

      --max-filesize "${MAX_TEXT_MB}M" \

      -g '!**/.git/objects/**' -g '!**/node_modules/**' -g '!**/.venv/**' \

      -g '!**/venv/**' -g '!**/Library/Caches/**' -g '!**/homebrew/Cellar/**' \

      -g '!*.jmod' -g '!*.jar' -g '!*.zip' -g '!*.tar*' -g '!*.dmg' -g '!*.pkg' \

      -e 'token|secret|api[_-]?key|password|passphrase|private[ _-]?key|BEGIN (RSA |OPENSSH |EC )?
PRIVATE KEY' \

      "$volume" > "$volume_out/sensitive_keyword_hits_redacted.txt" 2>> "$volume_out/rg_errors.log" || 
true



    rg --hidden --no-follow --no-messages --with-filename -i -n -o \

      --max-filesize "${MAX_TEXT_MB}M" \

      -g '!**/.git/objects/**' -g '!**/node_modules/**' -g '!**/.venv/**' \

      -g '!**/venv/**' -g '!**/Library/Caches/**' -g '!**/homebrew/Cellar/**' \

      -g '!*.jmod' -g '!*.jar' -g '!*.zip' -g '!*.tar*' -g '!*.dmg' -g '!*.pkg' \

      -e 'mdmBaseURL|axm-servicediscovery|com\.apple\.remotemanagement|ConfigurationProfiles|
RunAtLoad|KeepAlive|ProgramArguments|osascript|curl[[:space:]]|base64|cloudflare|warp|codex-local|
unsloth|gemma' \

      "$volume" > "$volume_out/forensic_keyword_hits.txt" 2>> "$volume_out/rg_errors.log" || true

  else

    printf 'ripgrep unavailable; text search not run\n' > "$volume_out/rg_errors.log"

  fi

}



summarize_directories() {

  volume_out=$1

  report="$volume_out/directory_summary.tsv"
```

```
  printf 'directory_group\tobjects\tcode_candidates\tparse_failures\tinvalid_or_unsigned_native_code\n' >
"$report"

  for dir in "$volume_out"/directories/*; do

    [ -d "$dir" ] || continue

    objects=$(awk 'NR>1{n++} END{print n+0}' "$dir/objects.tsv")

    candidates=$(awk 'NR>1{n++} END{print n+0}' "$dir/code_verification.tsv")

    parse_failures=$(awk -F '\t' 'NR>1 && $4=="fail"{n++} END{print n+0}' "$dir/code_verification.tsv")

    native_failures=$(awk -F '\t' 'NR>1 && ($5=="invalid" || $5=="unsigned"){n++} END{print n+0}' "$dir/
code_verification.tsv")

    printf '%s\t%s\t%s\t%s\t%s\n' "$(basename "$dir")" "$objects" "$candidates" "$parse_failures"
"$native_failures" >> "$report"

  done

}



inventory_volume() {

  volume=$1

  [ -d "$volume" ] || fatal "source volume is not a directory: $volume"

  volume="$(cd "$volume" && pwd -P)"

  case "$OUT/" in "$volume/"*) fatal "output directory cannot be inside source volume: $volume";; esac



  vname="$(basename "$volume")"

  vslug="$(safe_component "$vname")_$(path_id "$volume")"

  CURRENT_VOUT="$OUT/$vslug"

  export CURRENT_VOUT

  mkdir -p "$CURRENT_VOUT/directories"



  source_mount=$(df "$volume" 2>/dev/null | awk 'NR==2{print $NF}')
```

```
  mount_line=$(mount | grep -F " on $source_mount " | head -n 1 || true)
  disk_read_only=$(diskutil info "$source_mount" 2>/dev/null | awk -F: '/Volume Read-Only/{gsub(/^[ \t]
+/,"",$2); print $2; exit}')
  if [ "$ALLOW_WRITABLE" -ne 1 ]; then
    case "$mount_line $disk_read_only" in
      *read-only*|*'Yes (read-only mount flag set)'*) ;;
      *) fatal "refusing writable or unverified source: $volume" ;;
    esac
  fi

  log "Starting volume: $volume"
  date -u +%Y-%m-%dT%H:%M:%SZ > "$CURRENT_VOUT/00_utc_start.txt"
  printf '%s\n' "$mount_line" > "$CURRENT_VOUT/00_mount.txt"
  diskutil info "$source_mount" > "$CURRENT_VOUT/00_diskutil_info.txt" 2>&1 || true
  ls -laOe@ "$volume" > "$CURRENT_VOUT/00_root_listing.txt" 2>&1 || true

  file_count=0
  partial=0
  while IFS= read -r -d '' path; do
    rel=$(relative_path "$path" "$volume")
    top=$(top_name "$rel")
    top_dir=$(top_output_dir "$top")
    kind="other"
    description=""
    link_target=""
    sha=""
```

```
class="not_applicable"

if [ -L "$path" ]; then
  kind="symlink"
  link_target=$(readlink "$path" 2>/dev/null || true)
  description=$(file -b -h "$path" 2>/dev/null || true)
elif [ -d "$path" ]; then
  kind="directory"
  description="directory"
elif [ -f "$path" ]; then
  kind="file"
  file_count=$((file_count + 1))
  if [ "$LIMIT_FILES" -gt 0 ] && [ "$file_count" -gt "$LIMIT_FILES" ]; then
    partial=1
    break
  fi
  description=$(file -b "$path" 2>/dev/null || true)
  class=$(classify_file "$path" "$description")
  if should_hash "$class"; then
    sha=$(shasum -a 256 "$path" 2>/dev/null | awk '{print $1}')
    [ -n "$sha" ] || record_error "sha256" "$rel" "1" "hash failed"
  fi
fi

stat_fields=$(stat -f '%Sp|%u|%g|%z|%m|%B|%Sf' "$path" 2>/dev/null || true)
mode=$(printf '%s' "$stat_fields" | awk -F '|' '{print $1}')
```

```
    uid=$(printf '%s' "$stat_fields" | awk -F '|' '{print $2}')
    gid=$(printf '%s' "$stat_fields" | awk -F '|' '{print $3}')
    size=$(printf '%s' "$stat_fields" | awk -F '|' '{print $4}')
    mtime=$(printf '%s' "$stat_fields" | awk -F '|' '{print $5}')
    birth=$(printf '%s' "$stat_fields" | awk -F '|' '{print $6}')
    flags=$(printf '%s' "$stat_fields" | awk -F '|' '{print $7}')
    write_object_row "$top_dir/objects.tsv" "$rel" "$kind" "$mode" "$uid" "$gid" "$size" "$mtime" "$birth"
"$flags" "$sha" "$class" "$description" "$link_target"

    case "$class" in
      mach-o|native-library|shell|python|javascript|typescript|json|plist|executable-other)
        verify_code_file "$path" "$rel" "$class" "$sha" "$top_dir"
        ;;
    esac
  done < <(find "$volume" -xdev -print0 2>> "$CURRENT_VOUT/find_errors.log")

  printf 'file_limit=%s\nfiles_seen=%s\npartial=%s\n' "$LIMIT_FILES" "$file_count" "$partial" >
"$CURRENT_VOUT/run_scope.txt"
  verify_app_bundles "$volume" "$CURRENT_VOUT"
  verify_containers "$volume" "$CURRENT_VOUT"
  collect_high_signal_paths "$volume" "$CURRENT_VOUT"
  summarize_directories "$CURRENT_VOUT"

  date -u +%Y-%m-%dT%H:%M:%SZ > "$CURRENT_VOUT/00_utc_end.txt"
  log "Finished volume: $volume (files seen: $file_count, partial: $partial)"
  find "$CURRENT_VOUT" -type f ! -name output_hashes.sha256 -exec shasum -a 256 {} \; >
"$CURRENT_VOUT/output_hashes.sha256"
```

```
}

{
  echo "case=$CASE"
  echo "started_utc=$(date -u +%Y-%m-%dT%H:%M:%SZ)"
  echo "script=$0"
  echo "script_sha256=$(shasum -a 256 "$0" | awk '{print $1}')"
  echo "hash_mode=$HASH_MODE"
  echo "max_text_mb=$MAX_TEXT_MB"
  echo "limit_files=$LIMIT_FILES"
  echo "inspect_dmg_contents=$INSPECT_DMG_CONTENTS"
  echo "dmg_max_files=$DMG_MAX_FILES"
  echo "container_only=$CONTAINER_ONLY"
  echo "allow_writable=$ALLOW_WRITABLE"
  for source_volume in "${VOLUMES[@]-}"; do
    [ -n "$source_volume" ] && printf 'source=%s\n' "$source_volume"
  done
} > "$OUT/00_case_manifest.txt"

{
  bash --version | head -n 1
  shasum -a 256 "$0"
  file --version 2>&1 | head -n 1
  codesign --version 2>&1 || true
  spctl --version 2>&1 || true
```

```
  plutil -help 2>&1 | head -n 1
  rg --version 2>&1 | head -n 1 || true
  jq --version 2>&1 || true
  python3 --version 2>&1 || true
  node --version 2>&1 || true
} > "$OUT/00_tool_versions.txt"

log "Case output: $OUT"
if [ -n "$CONTAINER_ONLY" ]; then
  verify_container_only "$CONTAINER_ONLY"
else
  for volume in "${VOLUMES[@]}"; do
    inventory_volume "$volume"
  done
fi

RUN_COMPLETE=1
rm -f "$OUT/INCOMPLETE"
touch "$OUT/COMPLETE"
date -u +%Y-%m-%dT%H:%M:%SZ > "$OUT/00_utc_end.txt"
log "Collection complete: $OUT"
find "$OUT" -type f ! -name case_output_hashes.sha256 -exec shasum -a 256 {} \; > "$OUT/
case_output_hashes.sha256"
```